Advanced Analysis of Algorithms CC5031

☺ Divide and Conquer Part 2☹

Lectures 7-8

Dr. Salman Masih

Assistant Professor

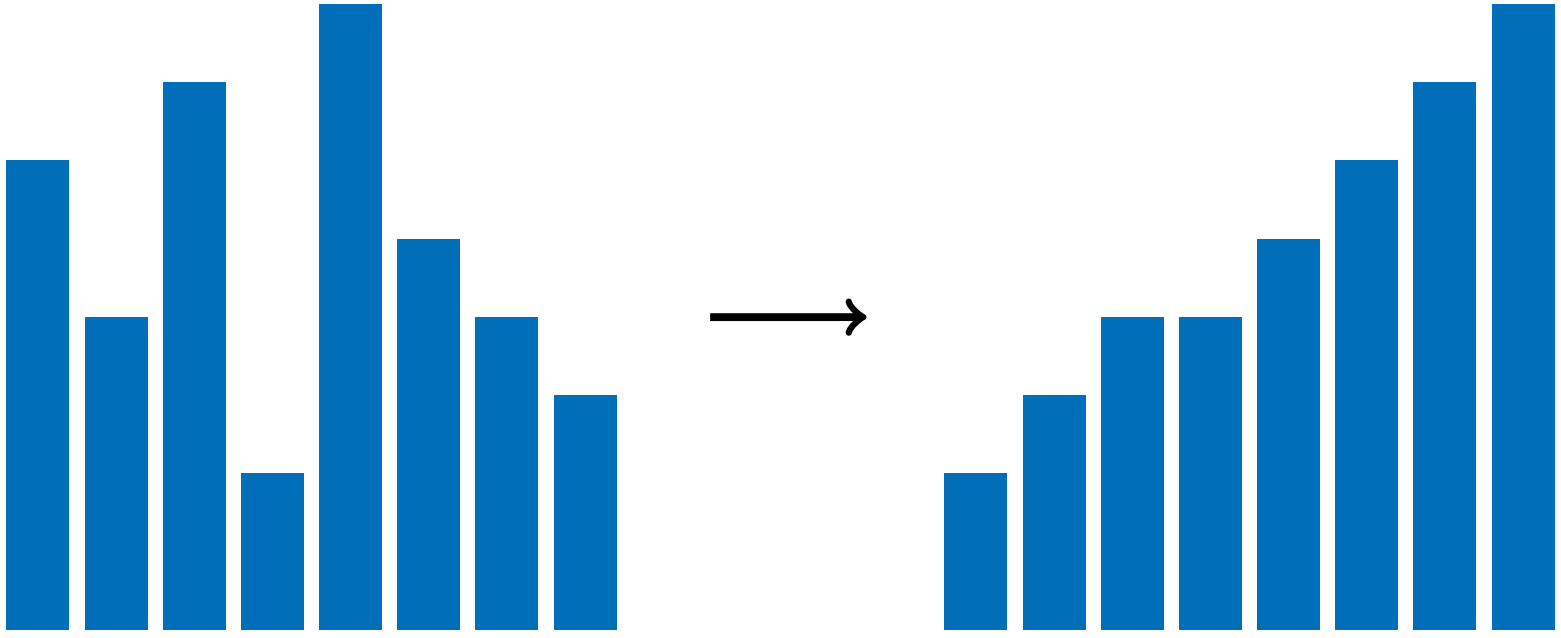
Department of Computer Science

University of Management and Technology, Sialkot, Pakistan

Outline

1. Problem Overview
2. Selection Sort
3. Merge Sort
4. Lower Bound for Comparison Based Sorting
5. Non-Comparison Based Sorting Algorithms

Sorting Problem



Sorting

Input: Sequence $A[1 \ldots n]$.

Output: Permutation $A'[1 \ldots n]$ of $A[1 \ldots n]$ in non-decreasing order.

Why Sorting?

- Sorting data is an important step of many efficient algorithms.

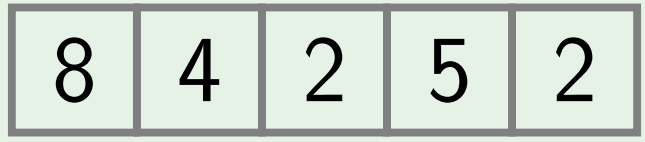
Why Sorting?

- Sorting data is an important step of many efficient algorithms.
- Sorted data allows for more efficient queries.

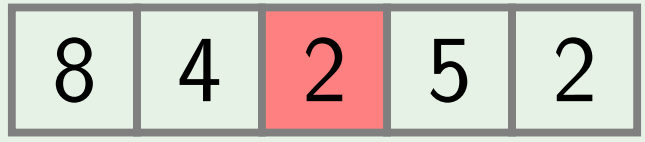
Outline

1. Problem Overview
2. Selection Sort
3. Merge Sort
4. Lower Bound for Comparison Based Sorting
5. Non-Comparison Based Sorting Algorithms

Selection sort: example

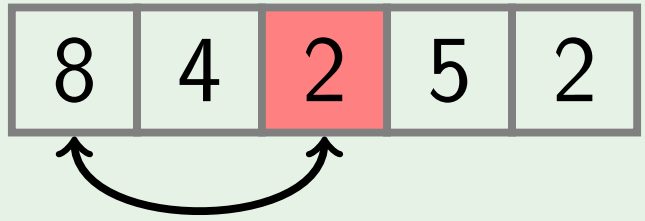


Selection sort: example



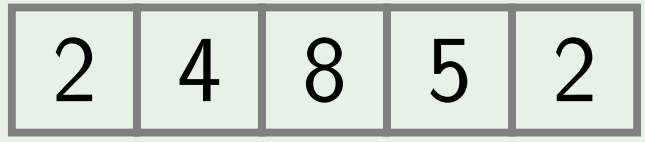
- Find a minimum by scanning the array

Selection sort: example



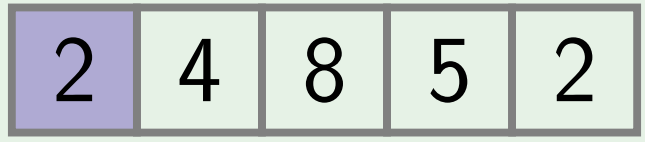
- Find a minimum by scanning the array
- Swap it with the first element

Selection sort: example



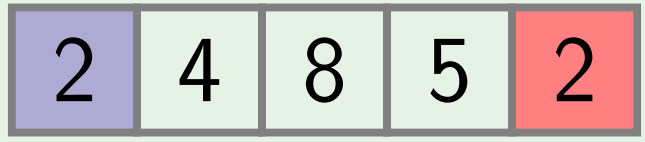
- Find a minimum by scanning the array
- Swap it with the first element

Selection sort: example



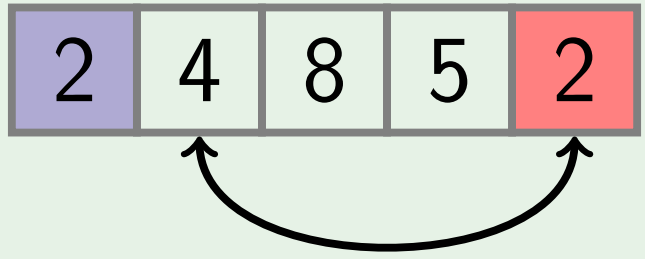
- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



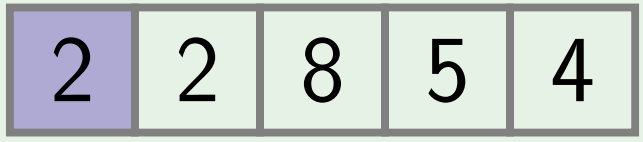
- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



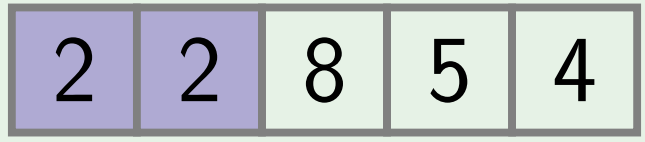
- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



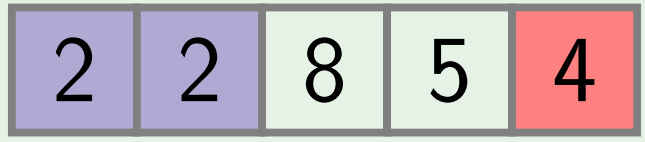
- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



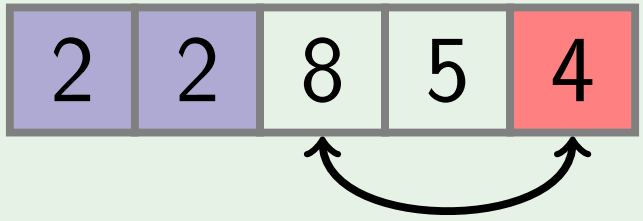
- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

Selection sort: example



- Find a minimum by scanning the array
- Swap it with the first element
- Repeat with the remaining part of the array

SelectionSort($A[1 \ldots n]$)

```
for i from 1 to n:
    minIndex ← i
    for j from i + 1 to n:
        if A[j] < A[minIndex]:
            minIndex ← j
    {A[minIndex] = min A[i ... n]}
    swap(A[i], A[minIndex])
    {A[1 ... i] is in final position}
```

SelectionSort($A[1 \ldots n]$)

```
for i from 1 to n:
    minIndex ← i
    for j from i + 1 to n:
        if A[j] < A[minIndex]:
            minIndex ← j
    {A[minIndex] = min A[i ... n]}
    swap(A[i], A[minIndex])
    {A[1 ... i] is in final position}
```

Online visualization: selection sort

Lemma

The running time of $\texttt{SelectionSort}(A[1 \ldots n])$ is $O(n^2)$.

Lemma

The running time of $\texttt{SelectionSort}(A[1 \ldots n])$ is $O(n^2)$.

Proof

n iterations of outer loop, at most n iterations of inner loop. $\square$

Too Pessimistic Estimate?

- As i grows, the number of iterations of the inner loop decreases: j iterates from $i + 1$ to n.

Too Pessimistic Estimate?

- As i grows, the number of iterations of the inner loop decreases: j iterates from $i + 1$ to n.
- A more accurate estimate for the total number of iterations of the inner loop is $(n - 1) + (n - 2) + \cdots + 1$.

Too Pessimistic Estimate?

- As i grows, the number of iterations of the inner loop decreases: j iterates from $i+1$ to n.
- A more accurate estimate for the total number of iterations of the inner loop is $(n-1)+(n-2)+\cdots+1$.
- We will show that this sum is $\Theta(n^2)$ implying that our initial estimate is actually tight.

Arithmetic Series

Lemma

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Arithmetic Series

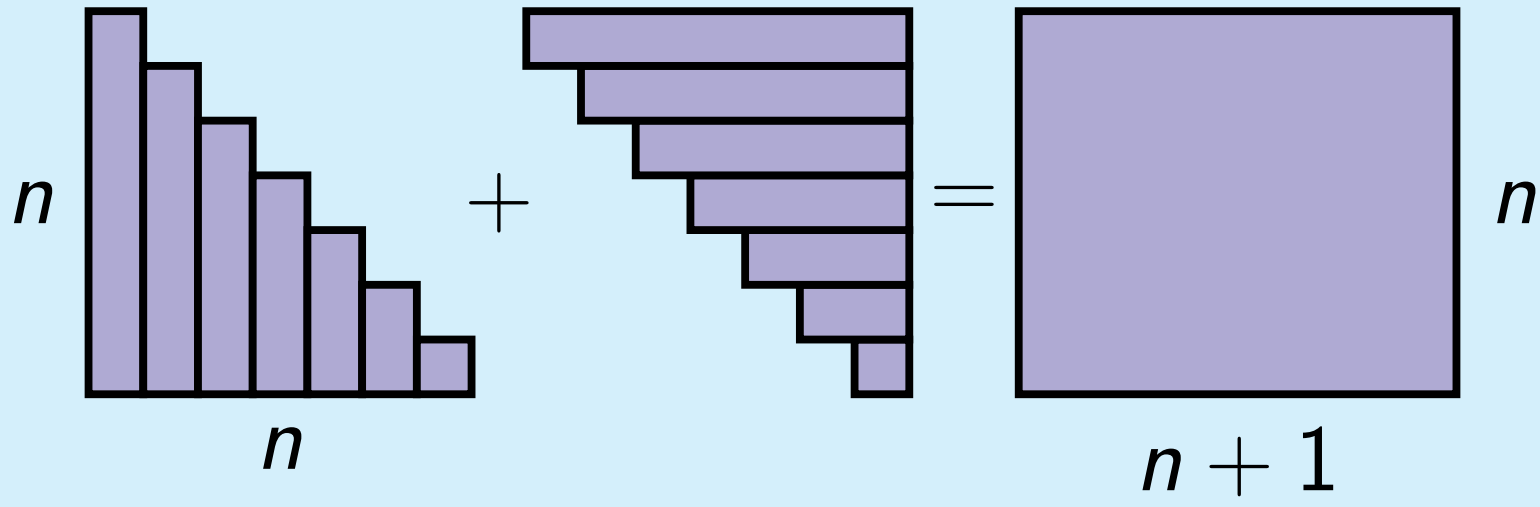
Lemma

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Proof

$$\begin{array}{cccc} 1 & 2 & \cdots & n \\ n & n-1 & \cdots & 1 \\ \hline n+1 & n+1 & \cdots & n+1 \end{array} = n(n+1) \quad \square$$

Alternative proof



□

Selection Sort: Summary

- Selection sort is an easy to implement algorithm with running time $O(n^2)$.

Selection Sort: Summary

- Selection sort is an easy to implement algorithm with running time $O(n^2)$.
- Sorts in place: requires a constant amount of extra memory.

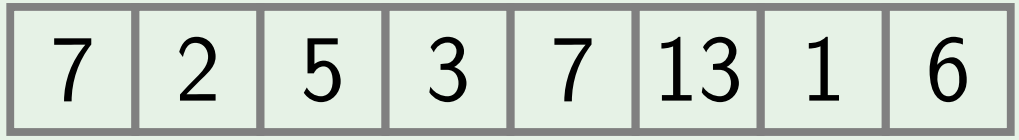
Selection Sort: Summary

- Selection sort is an easy to implement algorithm with running time $O(n^2)$.
- Sorts in place: requires a constant amount of extra memory.
- There are many other quadratic time sorting algorithms: e.g., insertion sort, bubble sort.

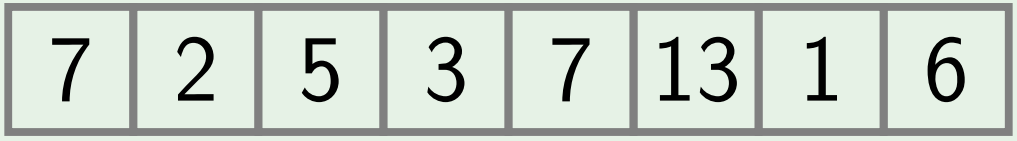
Outline

1. Problem Overview
2. Selection Sort
3. Merge Sort
4. Lower Bound for Comparison Based Sorting
5. Non-Comparison Based Sorting Algorithms

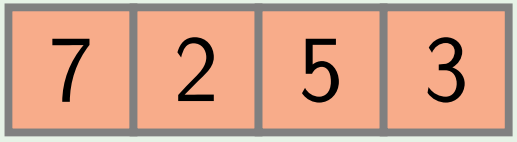
Example: merge sort

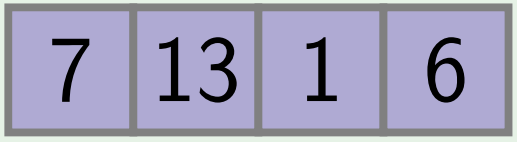


Example: merge sort

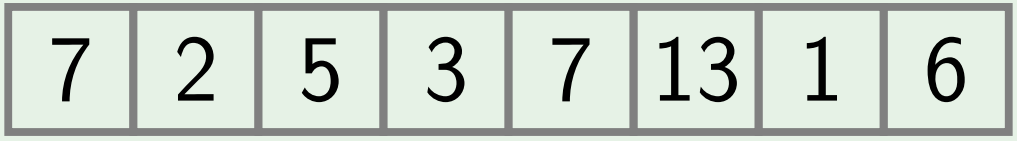


split the array into two halves

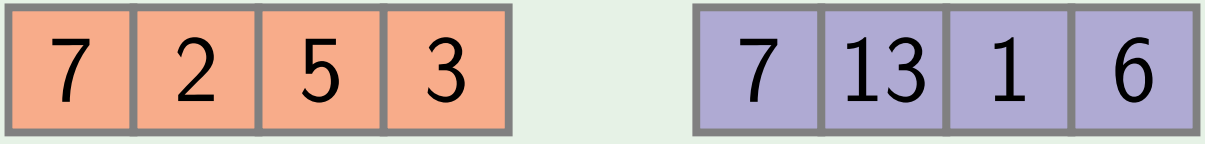




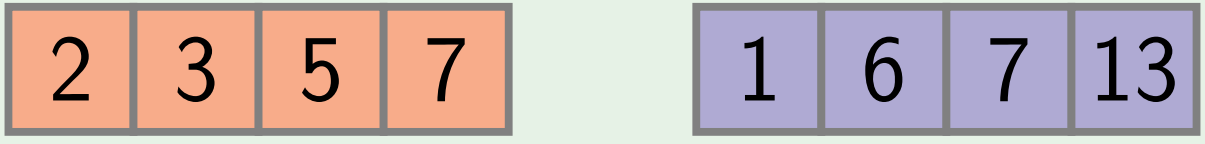
Example: merge sort



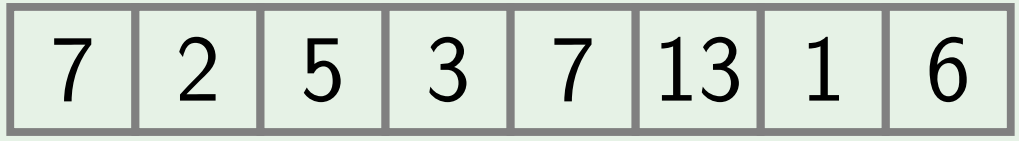
split the array into two halves



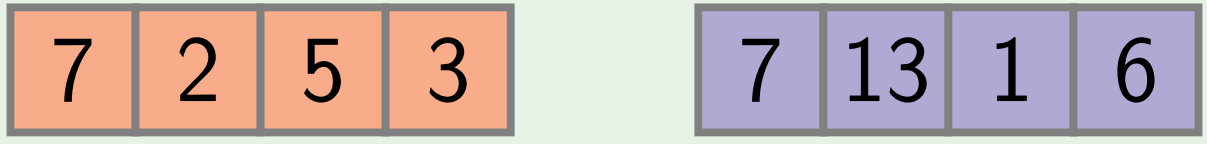
sort the halves recursively



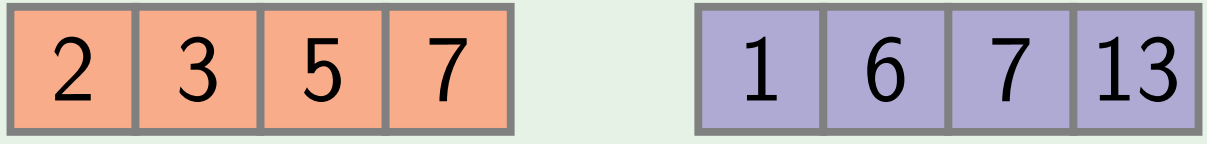
Example: merge sort



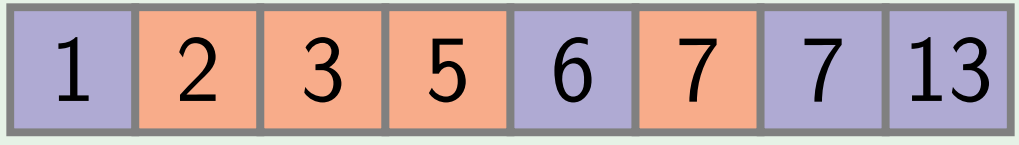
split the array into two halves



sort the halves recursively



merge the sorted halves into one array



MergeSort($A[1 \ldots n]$)

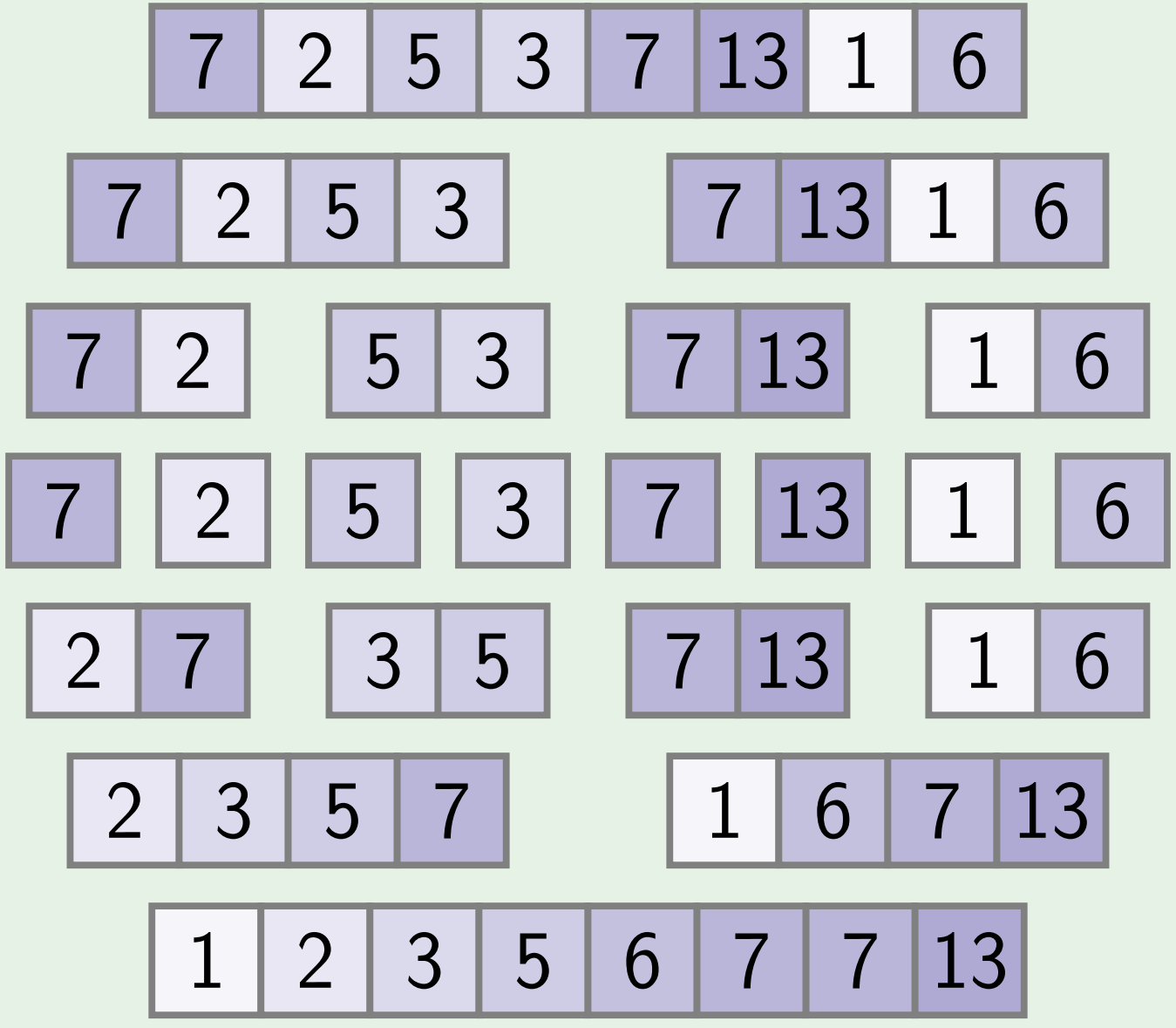
```
if n = 1:
  return A
m ← ⌊n/2⌋
B ← MergeSort(A[1 ... m])
C ← MergeSort(A[m + 1 ... n])
A' ← Merge(B, C)
return A'
```

Merging Two Sorted Arrays

Merge($B[1 \ldots p], C[1 \ldots q]$)

```
{B and C are sorted}
D ← empty array of size p + q
while B and C are both non-empty:
  b ← the first element of B
  c ← the first element of C
  if b ≤ c:
    move b from B to the end of D
  else:
    move c from C to the end of D
move the rest of B and C to the end of D
return D
```

Merge sort: example



Lemma

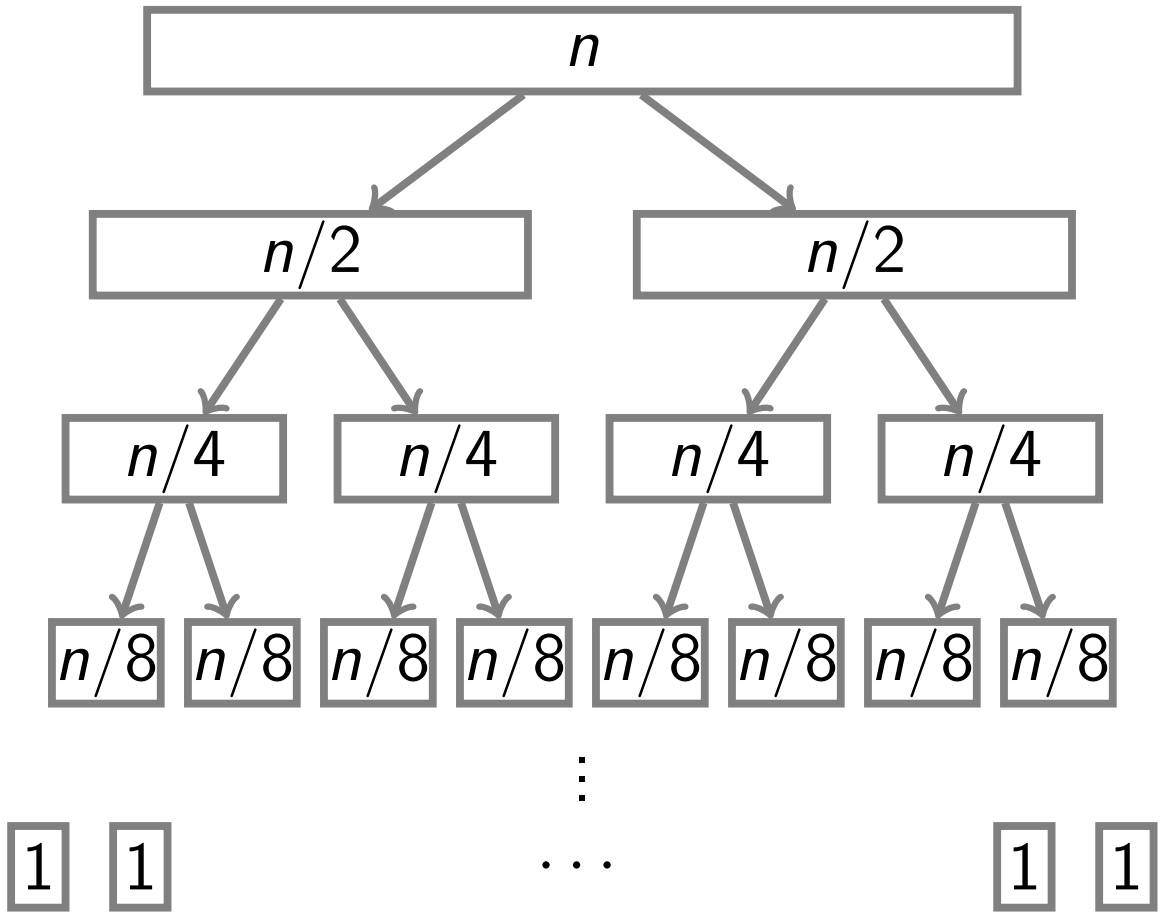
The running time of `MergeSort`$(A[1 \ldots n])$ is $O(n \log n)$.

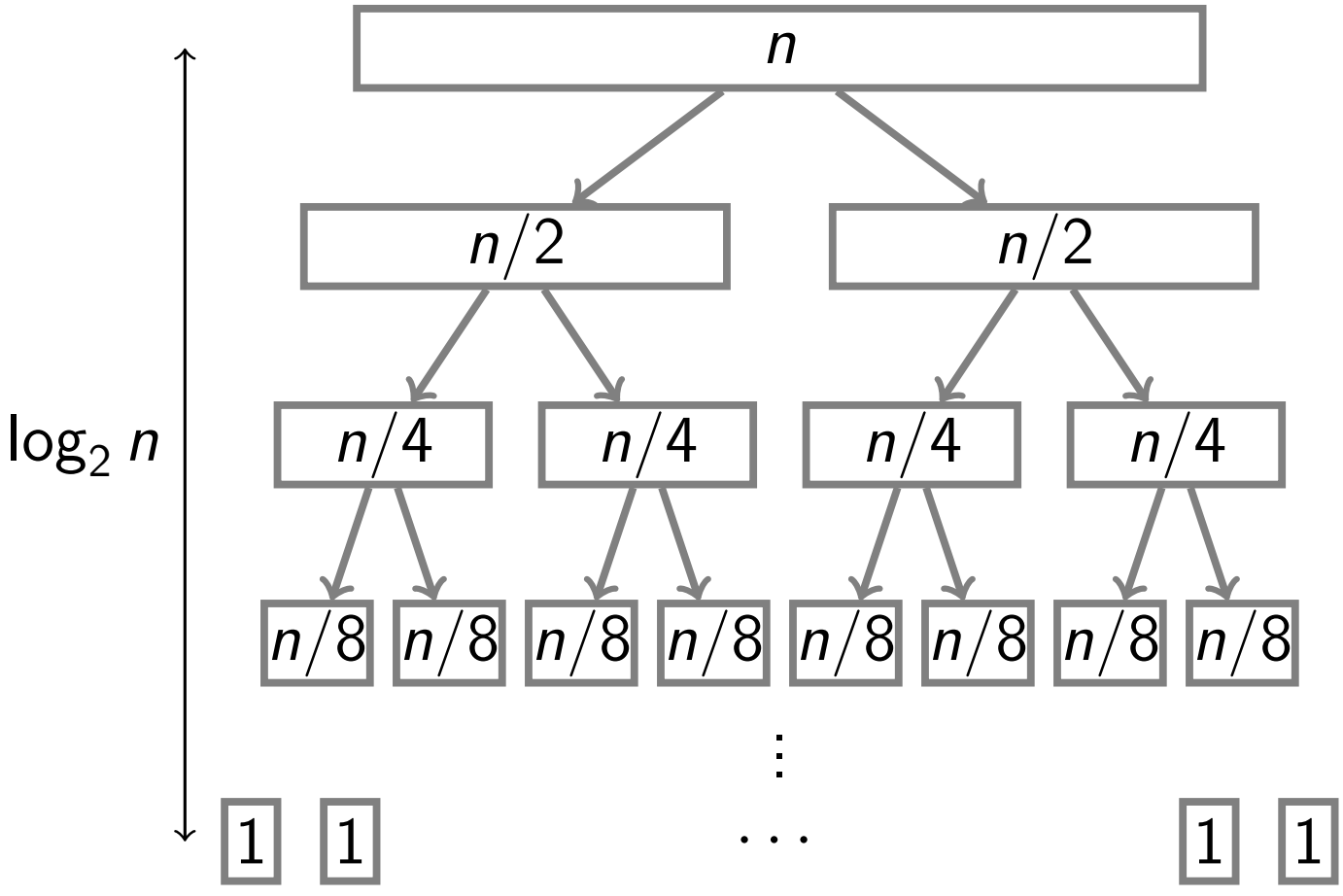
Lemma

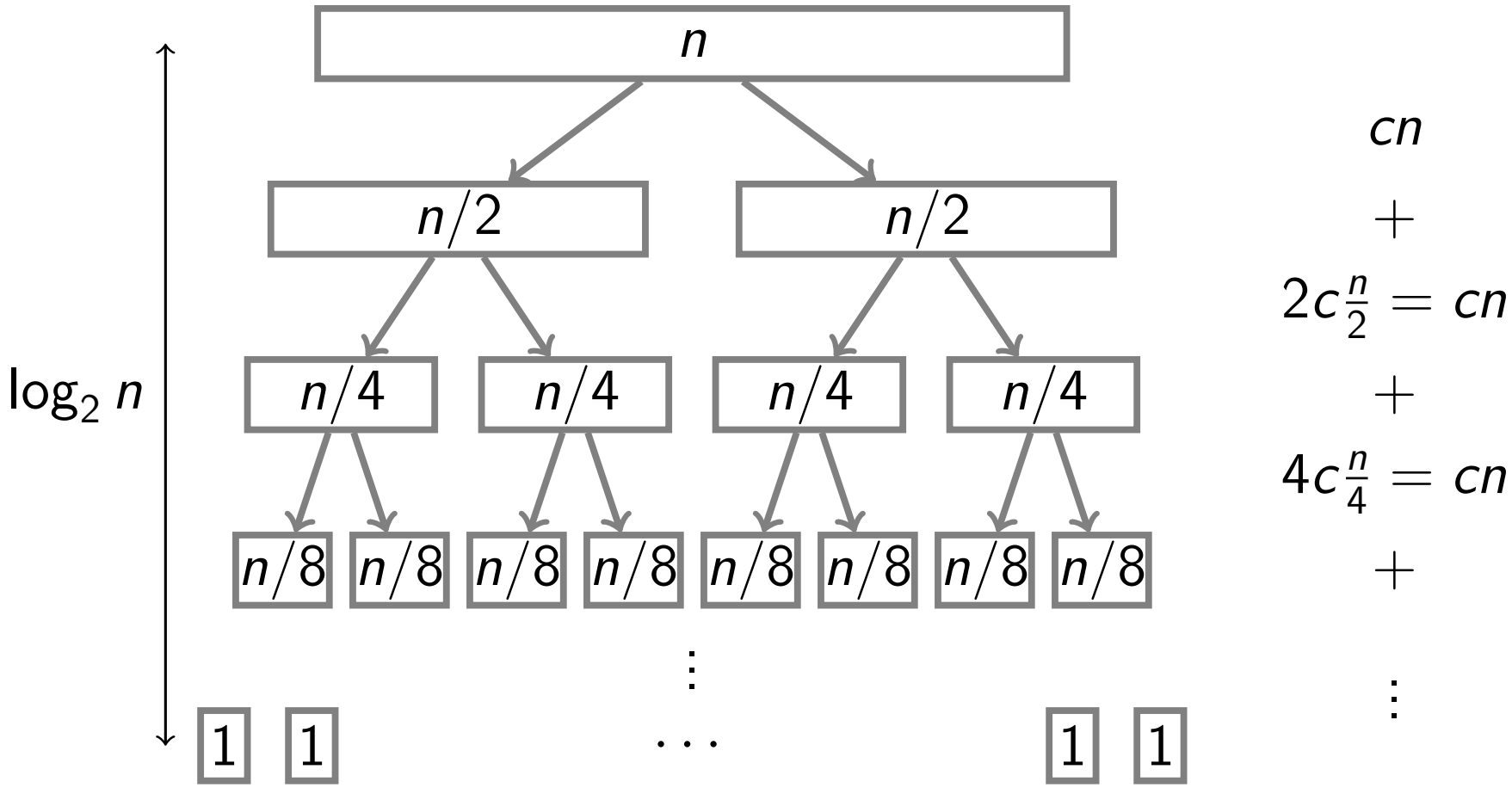
The running time of `MergeSort`$(A[1 \ldots n])$ is $O(n \log n)$.

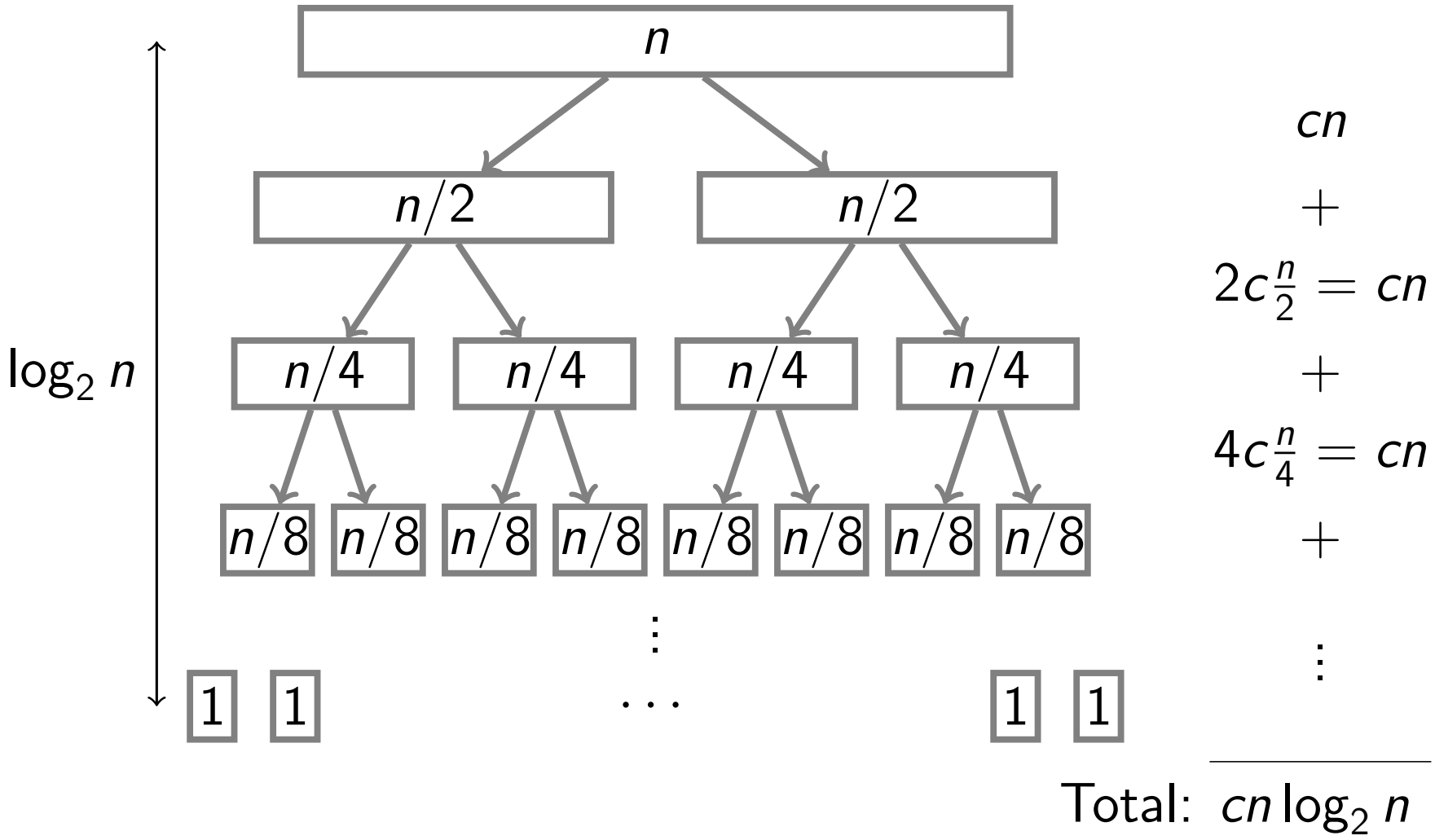
Proof

- The running time of merging B and C is $O(n)$.
- Hence the running time of `MergeSort`$(A[1 \ldots n])$ satisfies a recurrence $T(n) \leq 2T(n/2) + O(n)$.


n
n/2
n/2
n/4
n/4
n/4
n/4
n/8
n/8
n/8
n/8
n/8
n/8
n/8
n/8
1
1
1
1


n
n/2
n/2
n/4
n/4
n/4
n/4
n/8
n/8
n/8
n/8
n/8
n/8
n/8
n/8
1
1
1
1
log2 n


work:
n
n/2
n/2
n/4
n/4
n/4
n/4
n/8
n/8
n/8
n/8
n/8
n/8
n/8
n/8
1
1
1
1
log2 n
cn
+
2c n/2 = cn
+
4c n/4 = cn
+


work:
n
n/2
n/2
n/4
n/4
n/4
n/4
n/8
n/8
n/8
n/8
n/8
n/8
n/8
n/8
1
1
1
1
log2 n
cn
+
2c n/2 = cn
+
4c n/4 = cn
+
Total: cn log2 n

Outline

1. Problem Overview
2. Selection Sort
3. Merge Sort
4. Lower Bound for Comparison Based Sorting
5. Non-Comparison Based Sorting Algorithms

Definition

A comparison based sorting algorithm sorts objects by comparing pairs of them.

Definition

A comparison based sorting algorithm sorts objects by comparing pairs of them.

Example

Selection sort and merge sort are comparison based.

Lemma

Any comparison based sorting algorithm performs $\Omega(n \log n)$ comparisons in the worst case to sort n objects.

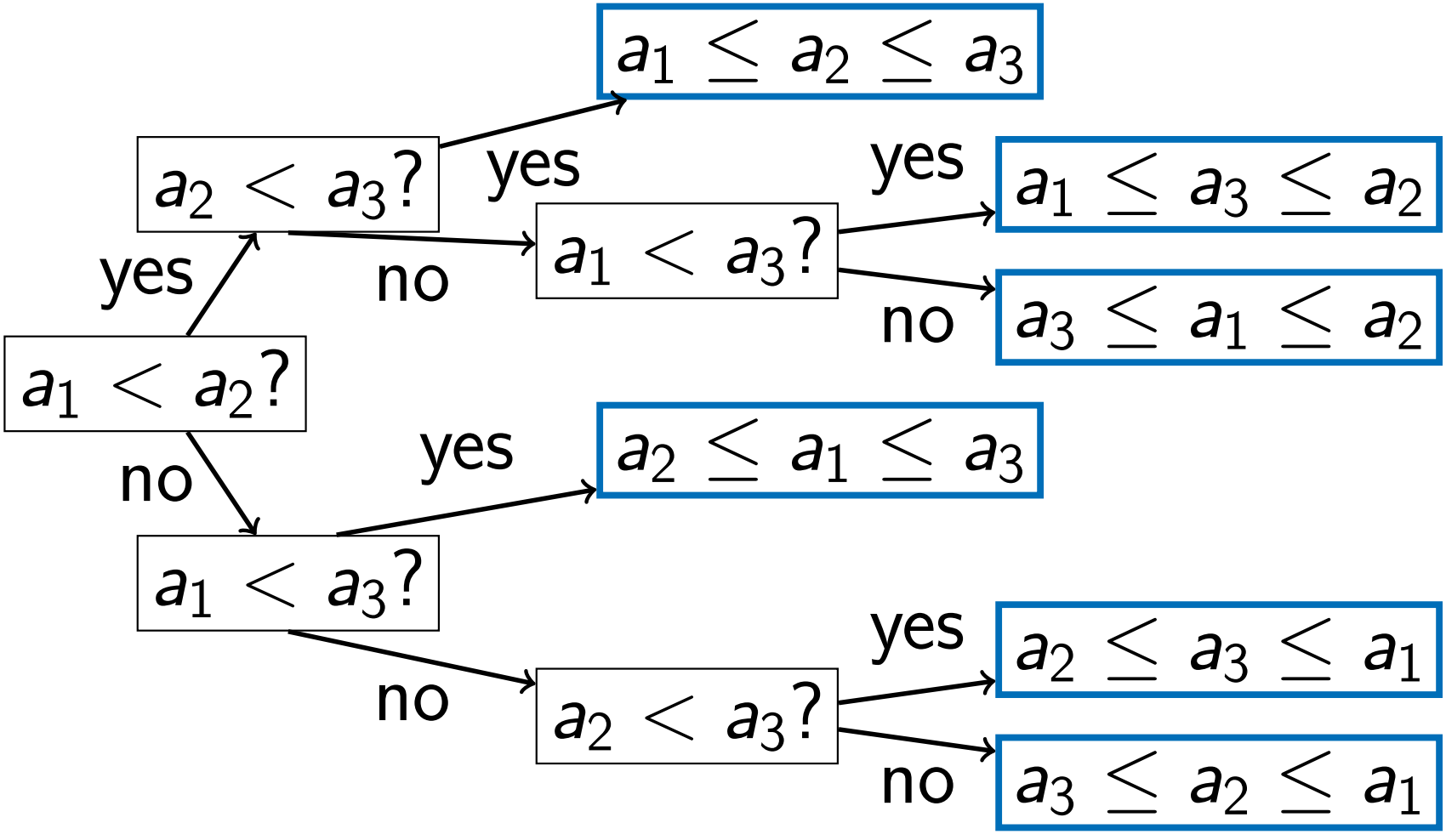
Lemma

Any comparison based sorting algorithm performs $\Omega(n \log n)$ comparisons in the worst case to sort n objects.

In other words

For any comparison based sorting algorithm, there exists an array $A[1 \ldots n]$ such that the algorithm performs at least $\Omega(n \log n)$ comparisons to sort A.

Decision Tree



Estimating Tree Depth

- the number of leaves ℓ in the tree must be at least $n!$ (the total number of permutations)

Estimating Tree Depth

- the number of leaves ℓ in the tree must be at least $n!$ (the total number of permutations)
- the worst-case running time of the algorithm (the number of comparisons made) is at least the depth d

Estimating Tree Depth

- the number of leaves ℓ in the tree must be at least $n!$ (the total number of permutations)
- the worst-case running time of the algorithm (the number of comparisons made) is at least the depth d
- $d \geq \log_2 \ell$ (or, equivalently, $2^d \geq \ell$)

Estimating Tree Depth

- the number of leaves ℓ in the tree must be at least $n!$ (the total number of permutations)
- the worst-case running time of the algorithm (the number of comparisons made) is at least the depth d
- $d \geq \log_2 \ell$ (or, equivalently, $2^d \geq \ell$)
- thus, the running time is at least

$$\log_2(n!) = \Omega(n \log n)$$

Lemma

$$\log_2(n!) = \Omega(n \log n)$$

Proof

$$\begin{aligned}
\log_2(n!) &= \log_2(1 \cdot 2 \cdot \cdots \cdot n) \\
&= \log_2 1 + \log_2 2 + \cdots + \log_2 n \\
&\geq \log_2 \frac{n}{2} + \cdots + \log_2 n \\
&\geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)
\end{aligned}$$



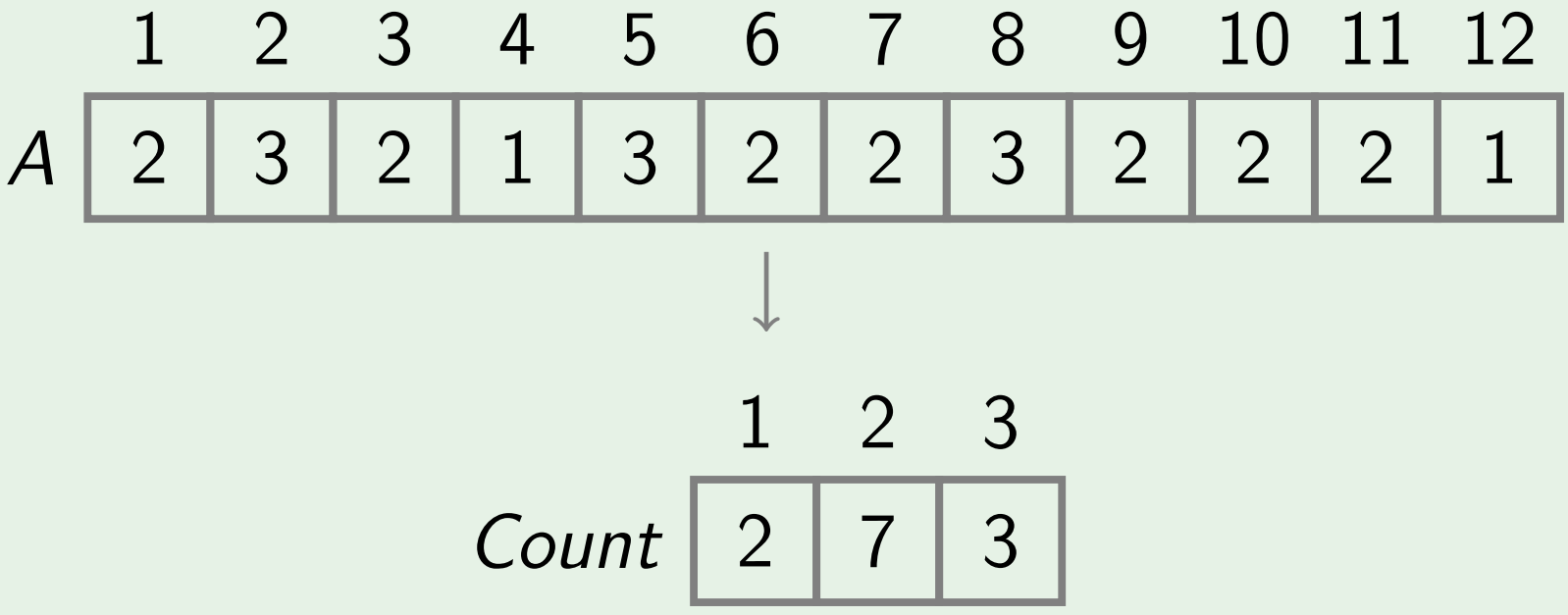
Outline

1. Problem Overview
2. Selection Sort
3. Merge Sort
4. Lower Bound for Comparison Based Sorting
5. Non-Comparison Based Sorting Algorithms

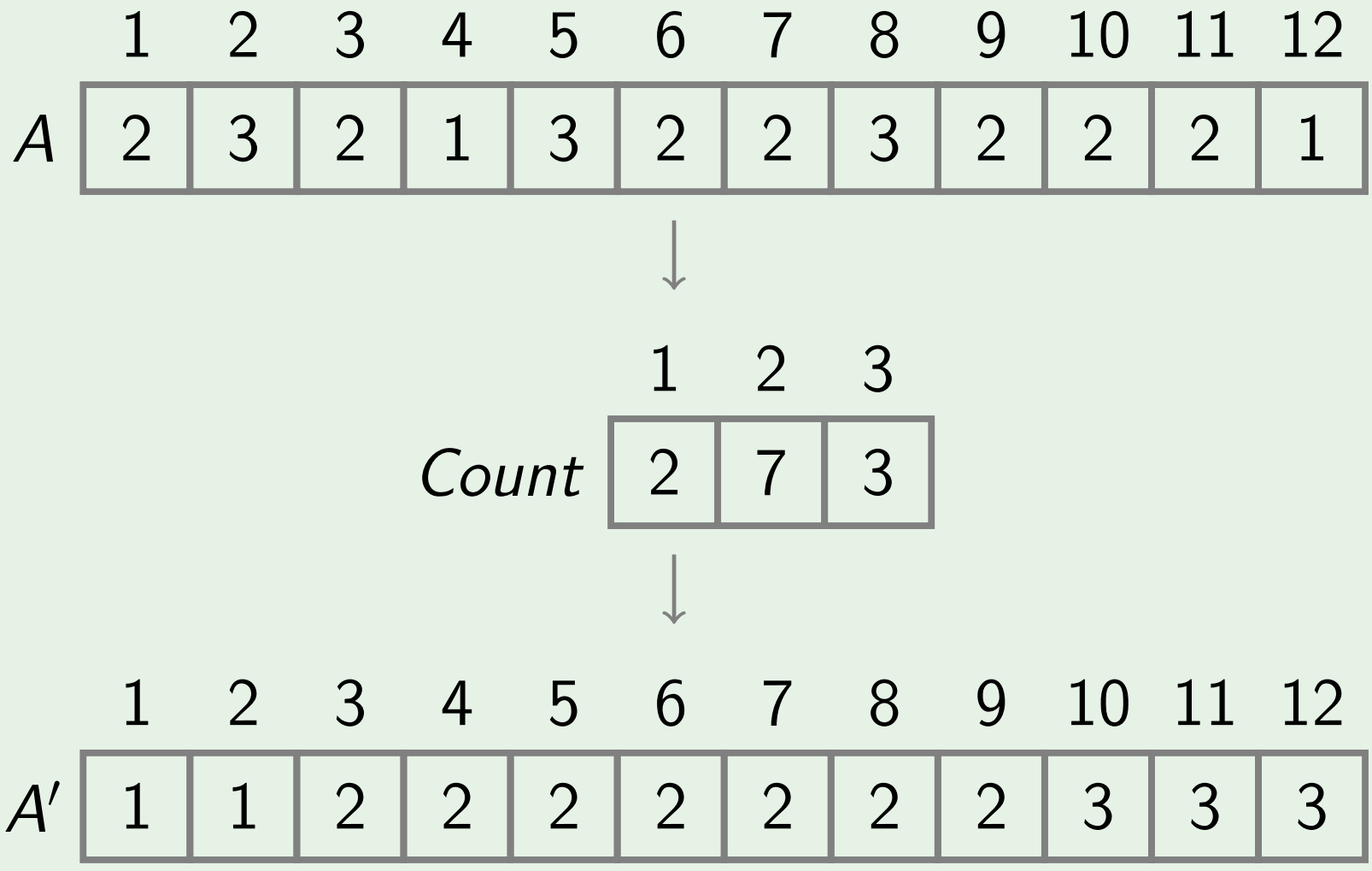
Example: sorting small integers

	1	2	3	4	5	6	7	8	9	10	11	12
A	2	3	2	1	3	2	2	3	2	2	2	1

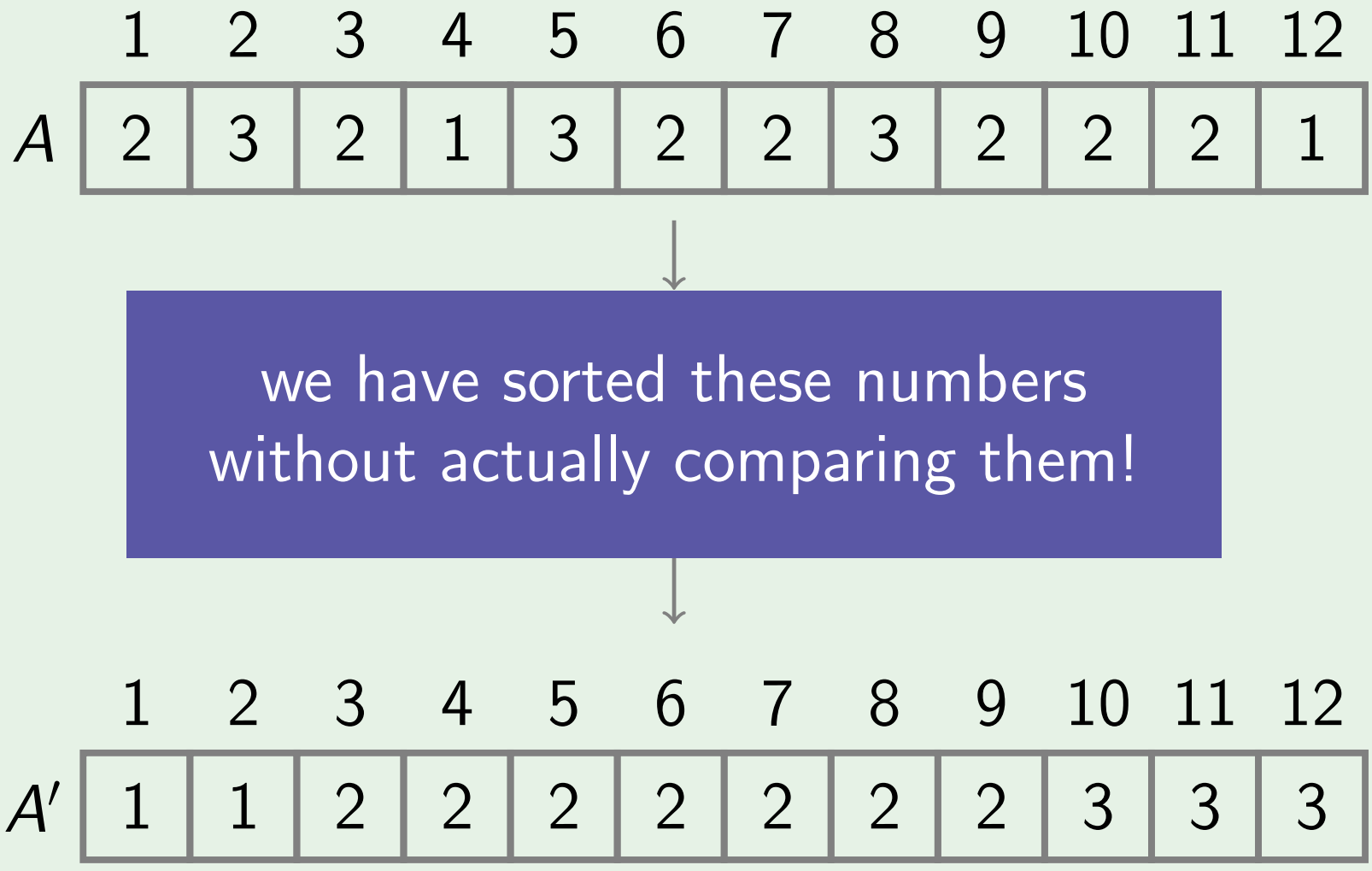
Example: sorting small integers



Example: sorting small integers



Example: sorting small integers



Counting Sort: Ideas

- Assume that all elements of $A[1 \ldots n]$ are integers from 1 to M.

Counting Sort: Ideas

- Assume that all elements of $A[1 \ldots n]$ are integers from 1 to M.
- By a single scan of the array A, count the number of occurrences of each $1 \leq k \leq M$ in the array A and store it in $Count[k]$.

Counting Sort: Ideas

- Assume that all elements of $A[1 \ldots n]$ are integers from 1 to M.
- By a single scan of the array A, count the number of occurrences of each $1 \leq k \leq M$ in the array A and store it in $Count[k]$.
- Using this information, fill in the sorted array A'.

CountSort($A[1 \ldots n]$)

$Count[1 \ldots M] \leftarrow [0, \ldots, 0]$
`for` i `from 1 to` n`:`
  $Count[A[i]] \leftarrow Count[A[i]] + 1$
{k `appears` $Count[k]$ `times in` A}
$Pos[1 \ldots M] \leftarrow [0, \ldots, 0]$
$Pos[1] \leftarrow 1$
`for` j `from 2 to` M`:`
  $Pos[j] \leftarrow Pos[j-1] + Count[j-1]$
{k `will occupy range` $[Pos[k] \ldots Pos[k+1]-1]$}
`for` i `from 1 to` n`:`
  $A'[Pos[A[i]]] \leftarrow A[i]$
  $Pos[A[i]] \leftarrow Pos[A[i]] + 1$

Lemma

Provided that all elements of $A[1 \ldots n]$ are integers from 1 to M, `CountSort`(A) sorts A in time $O(n + M)$.

Lemma

Provided that all elements of $A[1 \ldots n]$ are integers from 1 to M, `CountSort`(A) sorts A in time $O(n + M)$.

Remark

If $M = O(n)$, then the running time is $O(n)$.

Summary

- Merge sort uses the divide-and-conquer strategy to sort an n-element array in time $O(n \log n)$.
- No comparison based algorithm can do this (asymptotically) faster.
- One can do faster if something is known about the input array in advance (e.g., it contains small integers).

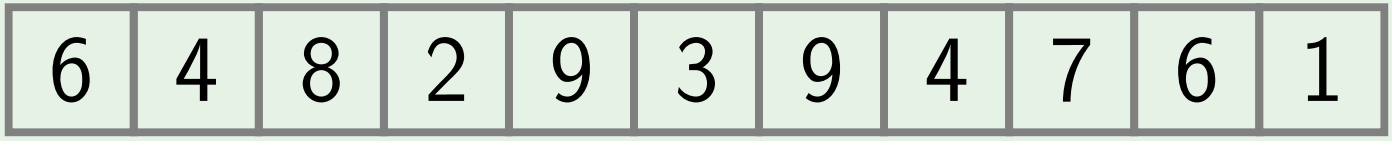
Outline

1. Overview
2. Algorithm
3. Random Pivot
4. Running Time Analysis
5. Equal Elements
6. Final Remarks

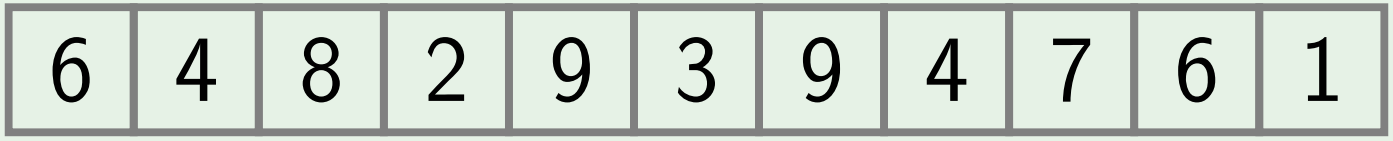
Quick Sort

- comparison based algorithm
- running time: $O(n \log n)$ (on average)
- efficient in practice

Example: quick sort



Example: quick sort



partition with respect to $x = A[1]$

in particular, x is in its final position



≤ 6 > 6

Example: quick sort

6	4	8	2	9	3	9	4	7	6	1

partition with respect to $x = A[1]$

in particular, x is in its final position

1	4	2	3	4	6	6	9	7	8	9

sort the two parts recursively

1	2	3	4	4	6	6	7	8	9	9

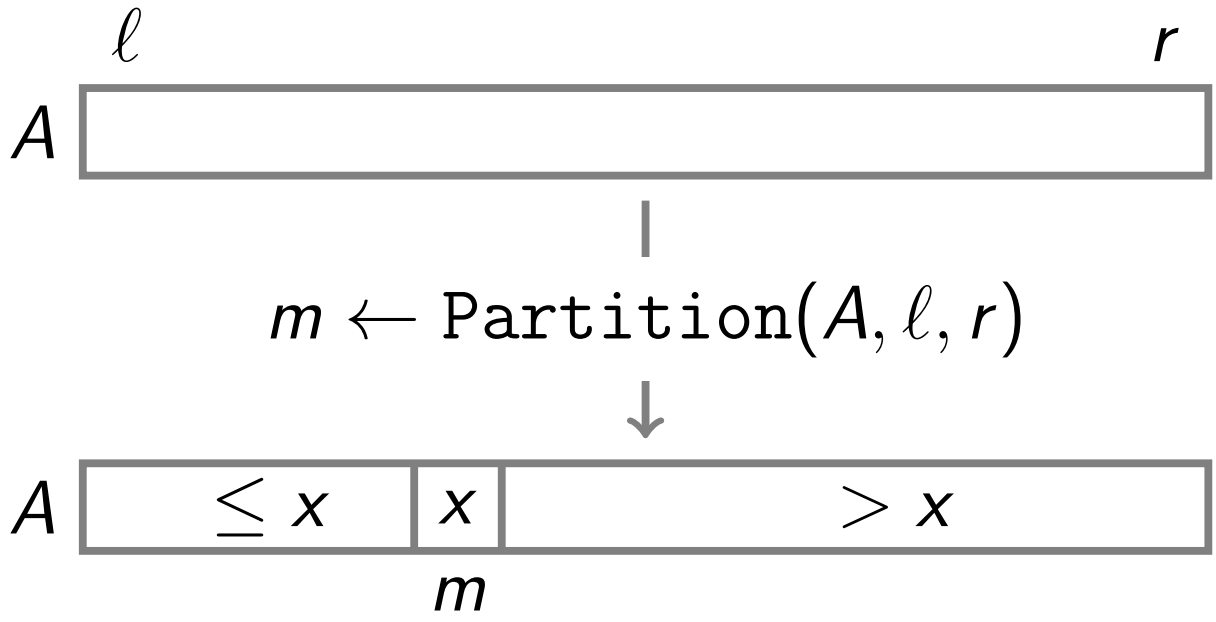
Outline

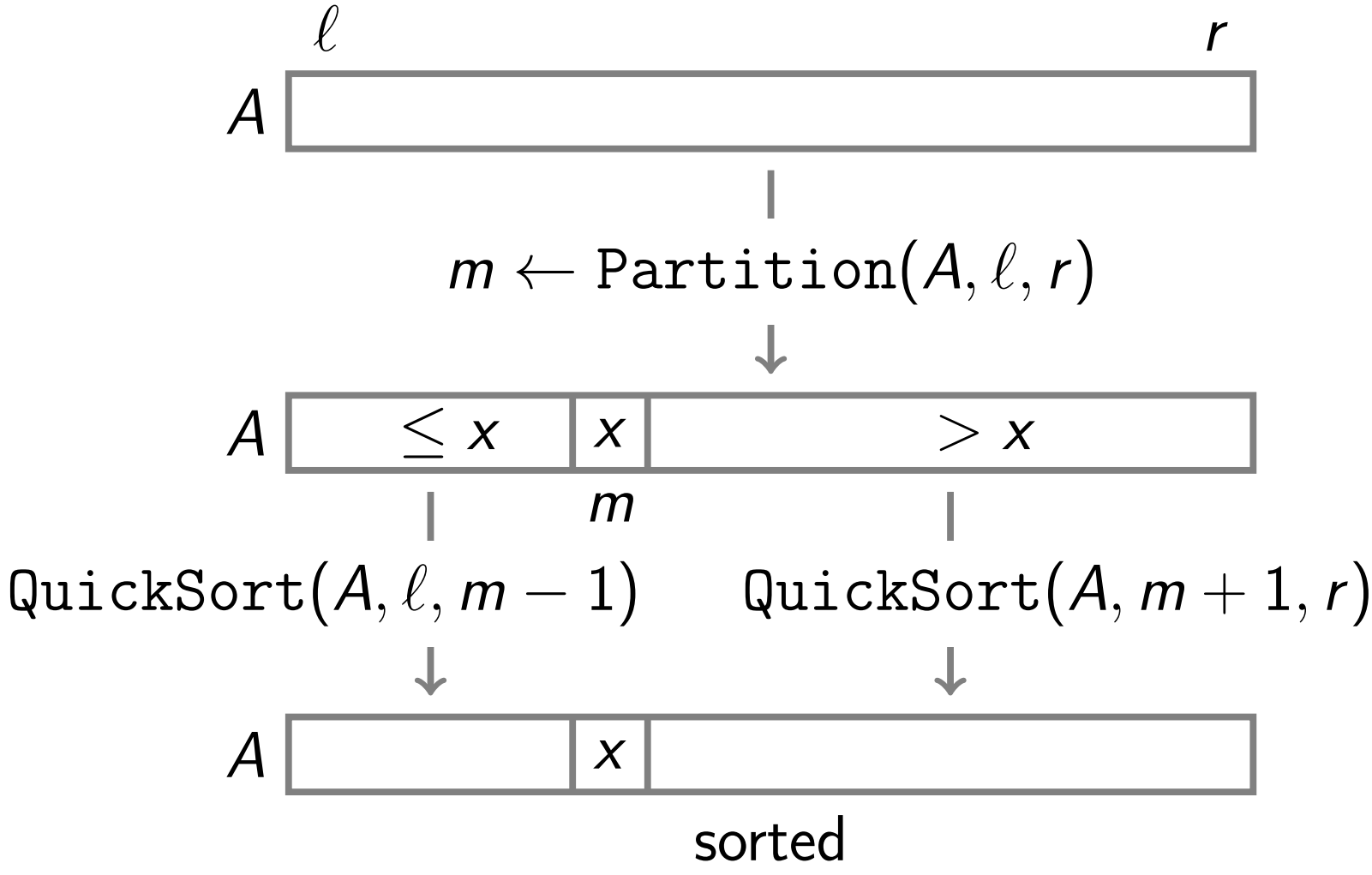
1. Overview
2. Algorithm
3. Random Pivot
4. Running Time Analysis
5. Equal Elements
6. Final Remarks

QuickSort(A, ℓ, r)

```
if ℓ ≥ r:
   return
m ← Partition(A, ℓ, r)
{A[m] is in the final position}
QuickSort(A, ℓ, m − 1)
QuickSort(A, m + 1, r)
```


ℓ
r
A


ℓ
r
A
m ← Partition(A, ℓ, r)
A
≤ x
x
> x
m


ℓ
r
A
$m \leftarrow \texttt{Partition}(A, \ell, r)$
A
$\leq x$
x
$> x$
m
$\texttt{QuickSort}(A, \ell, m-1)$
$\texttt{QuickSort}(A, m+1, r)$
A
x
sorted

Partitioning: example

- the pivot is $x = A[\ell]$

Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:

Partitioning: example

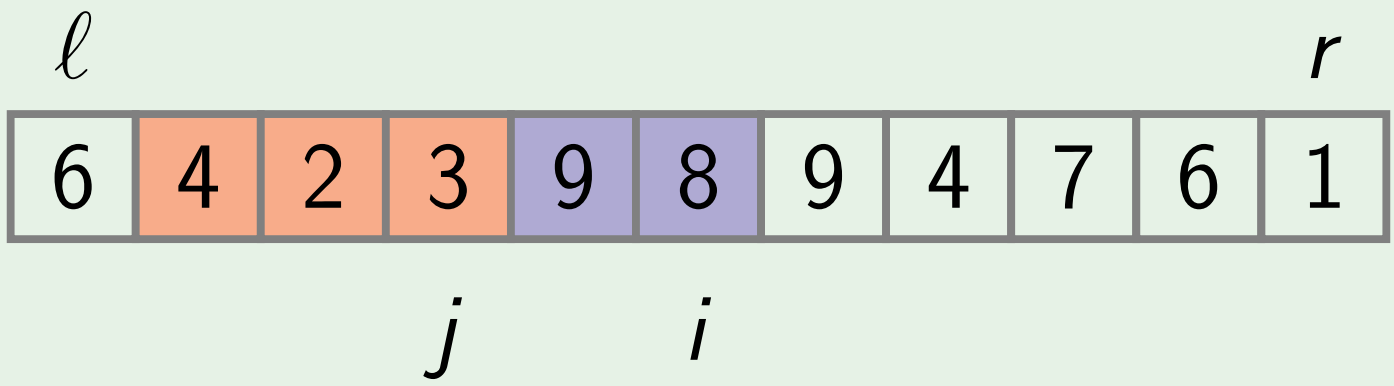
- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$

Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$

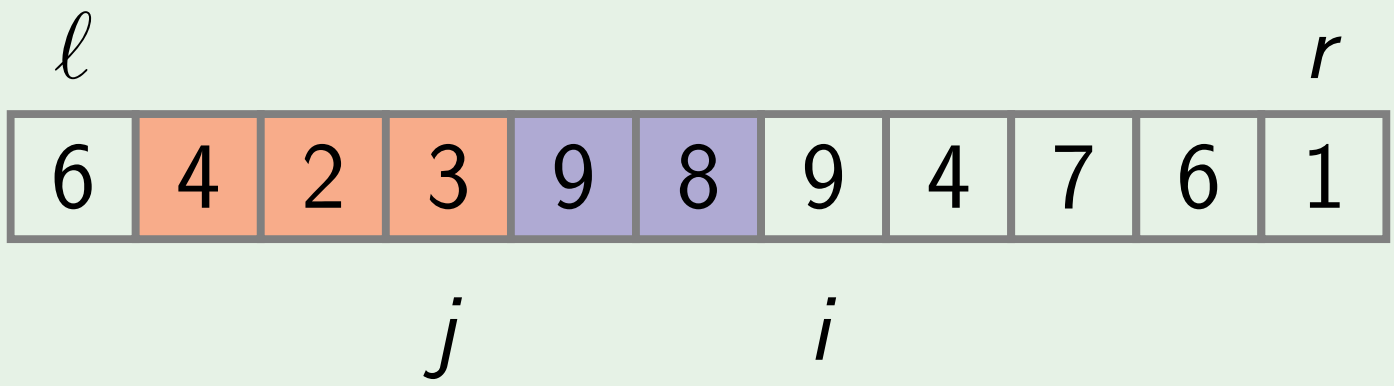
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



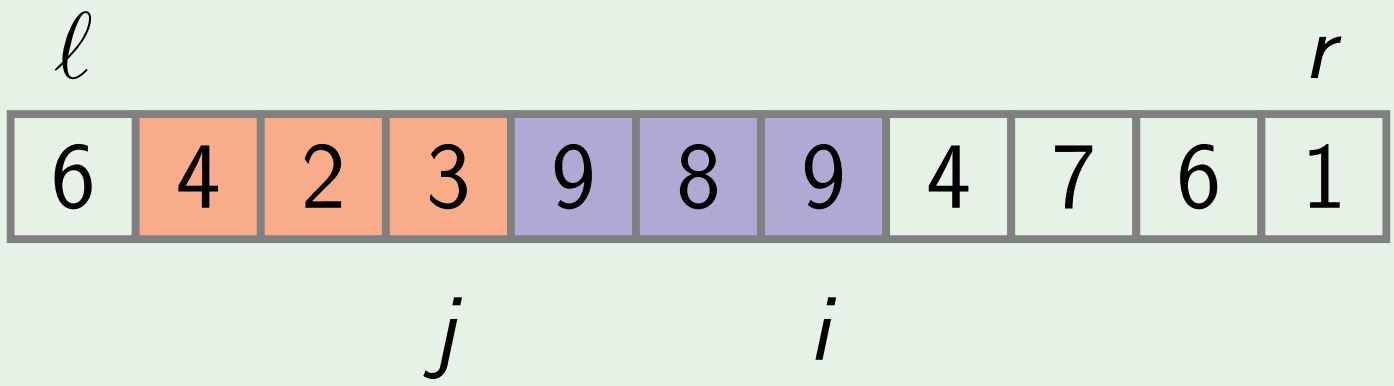
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



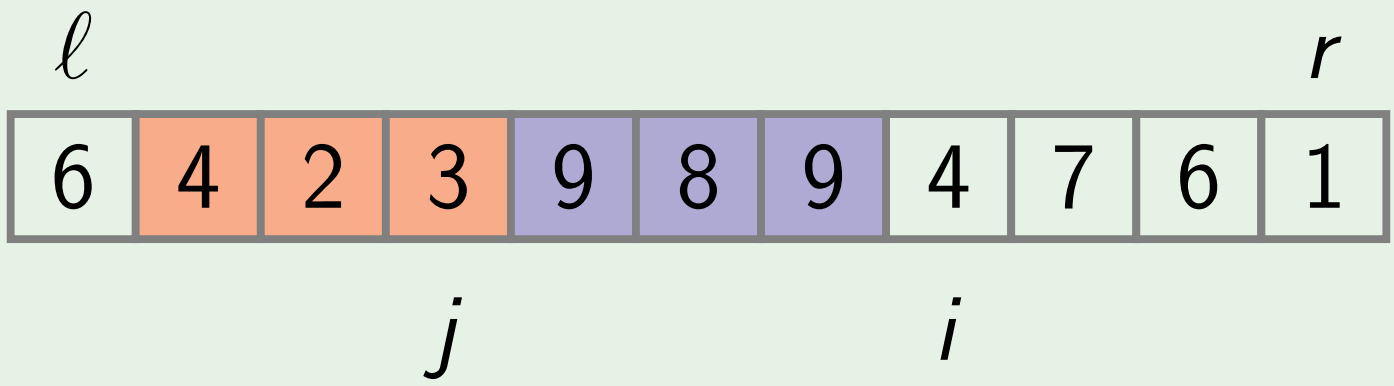
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



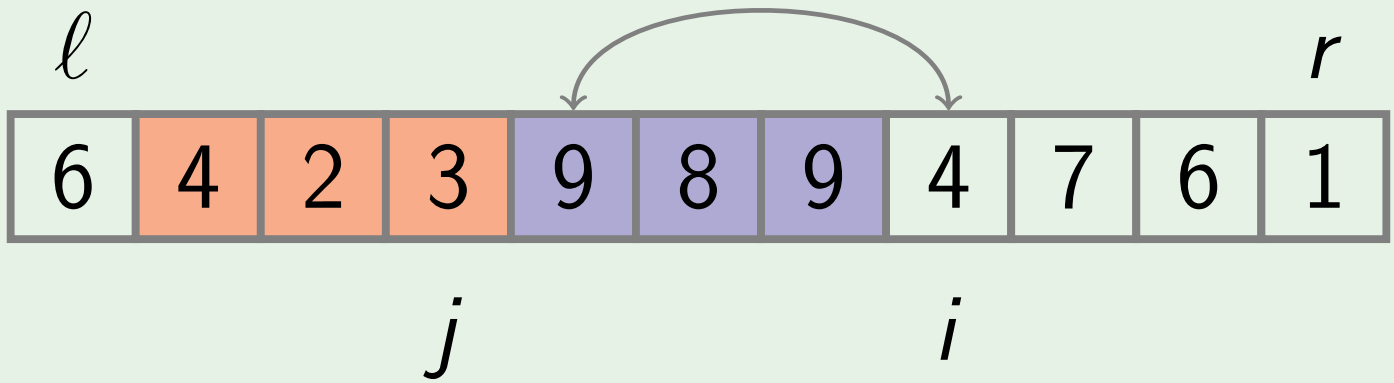
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



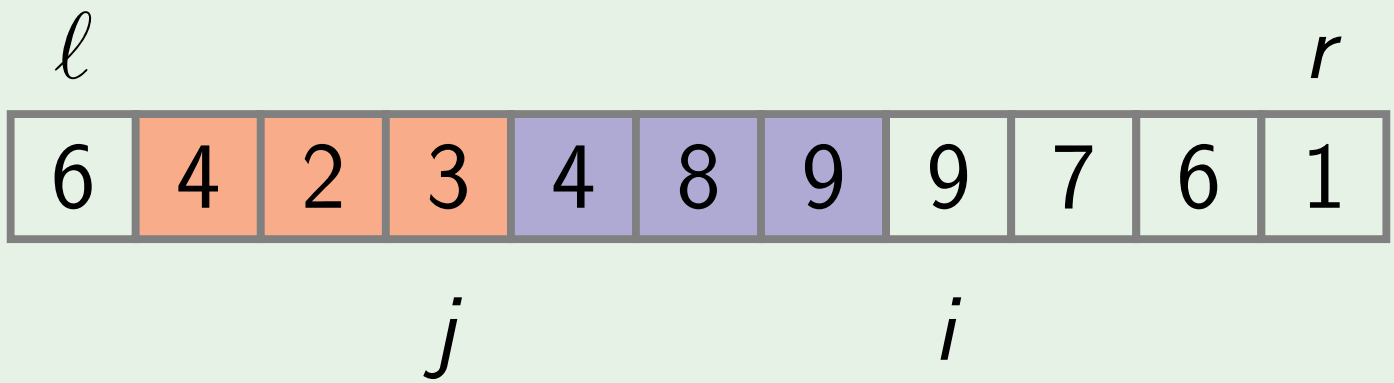
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



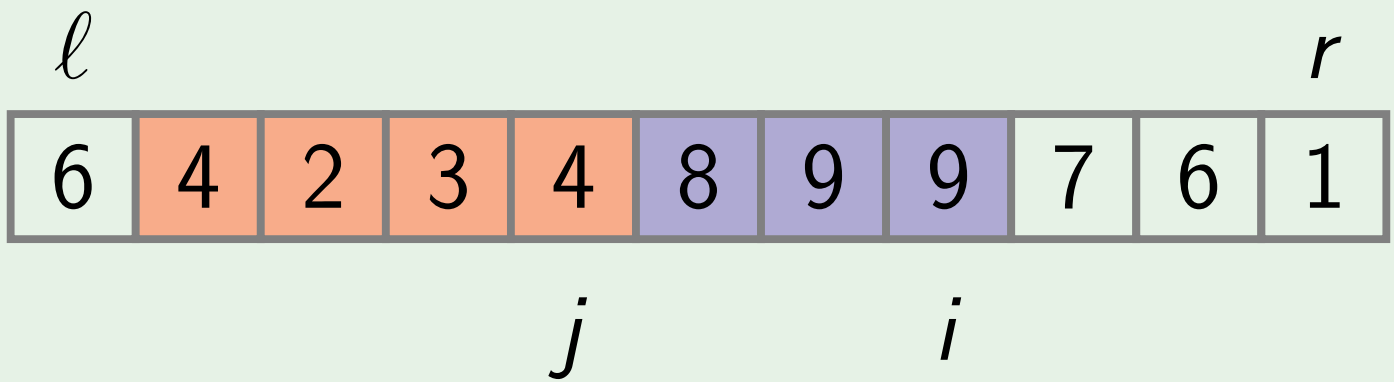
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



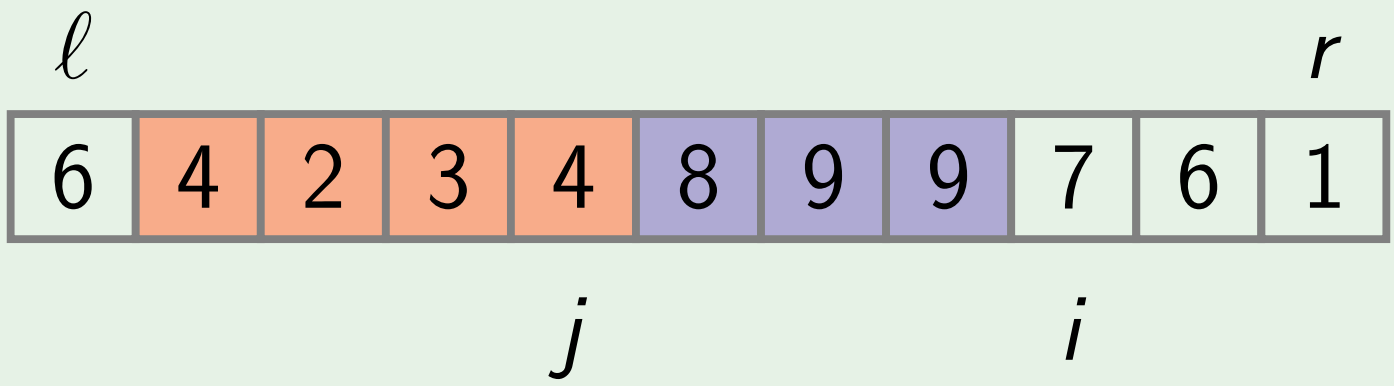
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



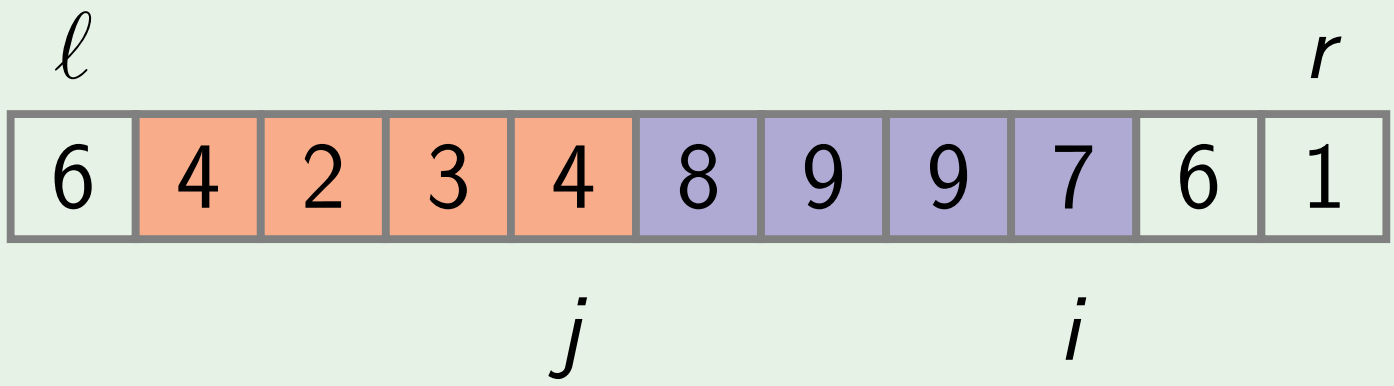
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



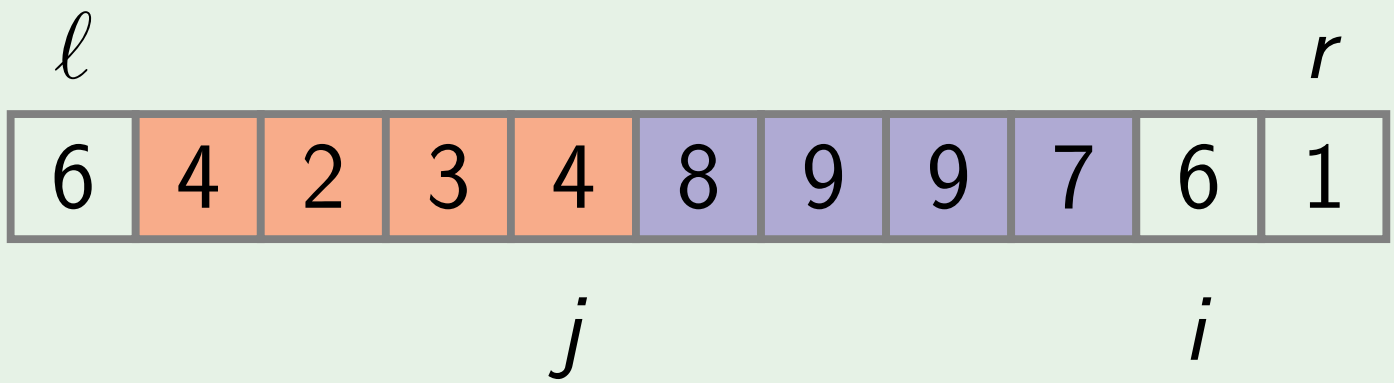
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



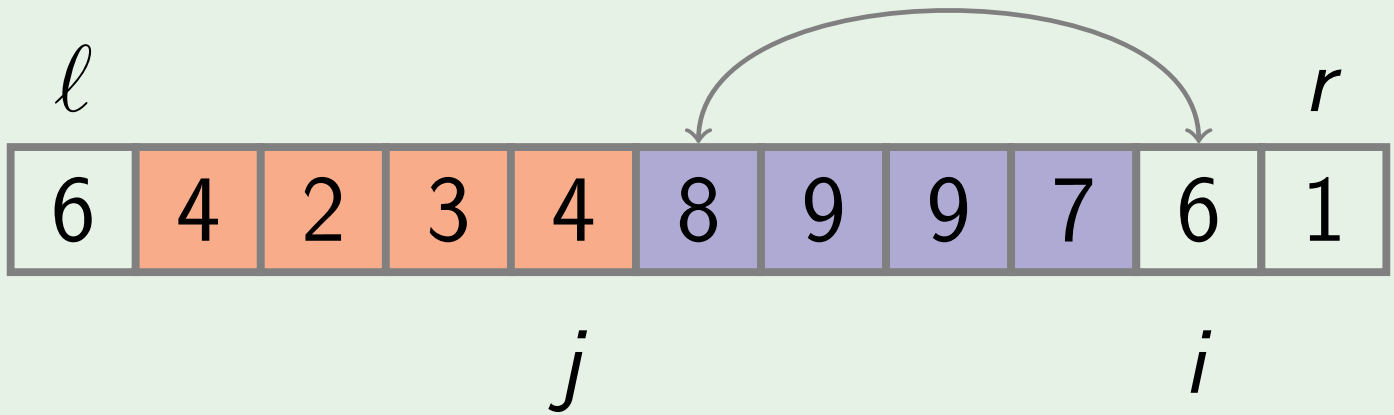
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



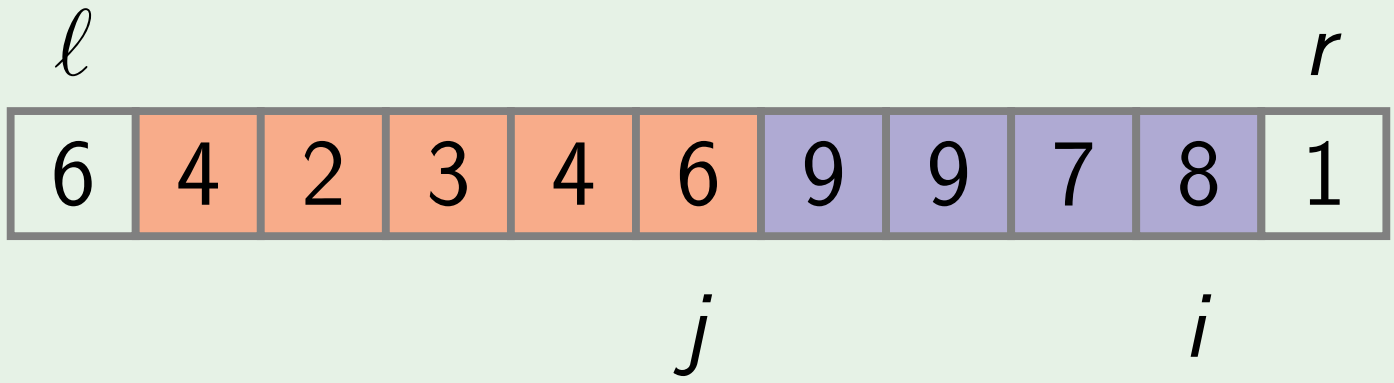
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



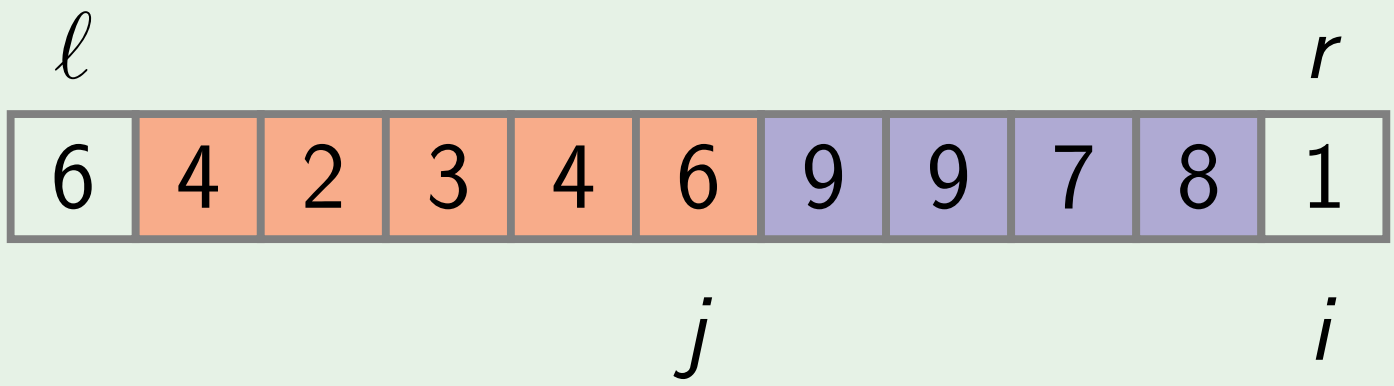
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



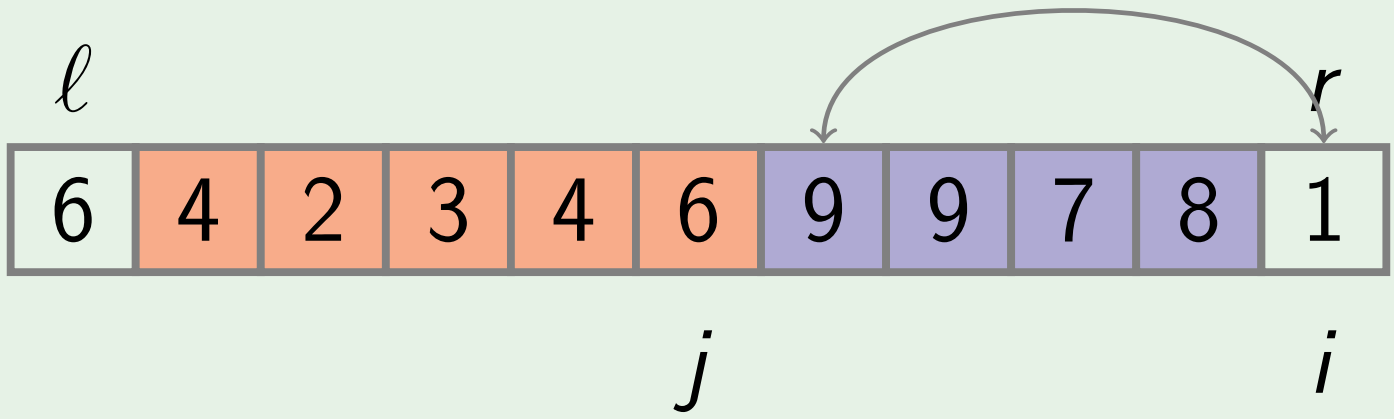
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



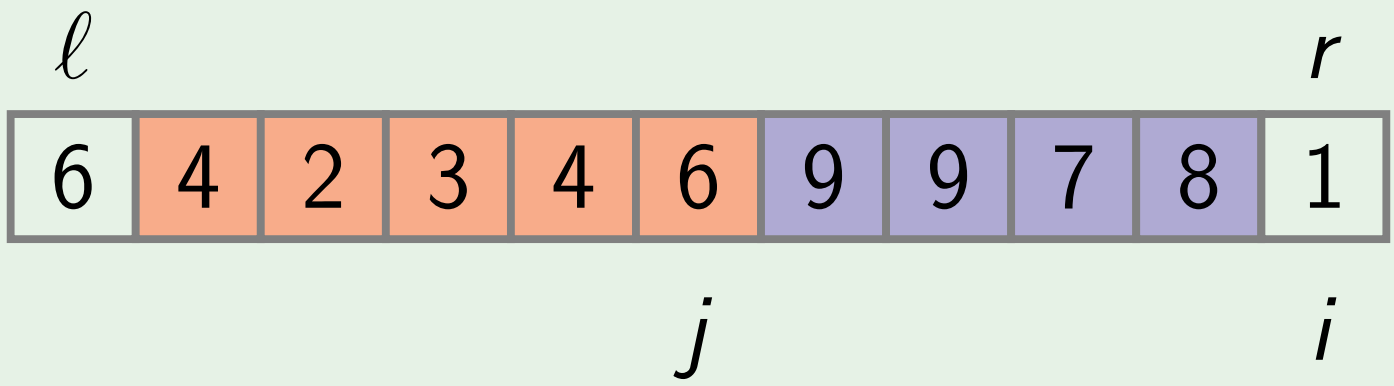
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



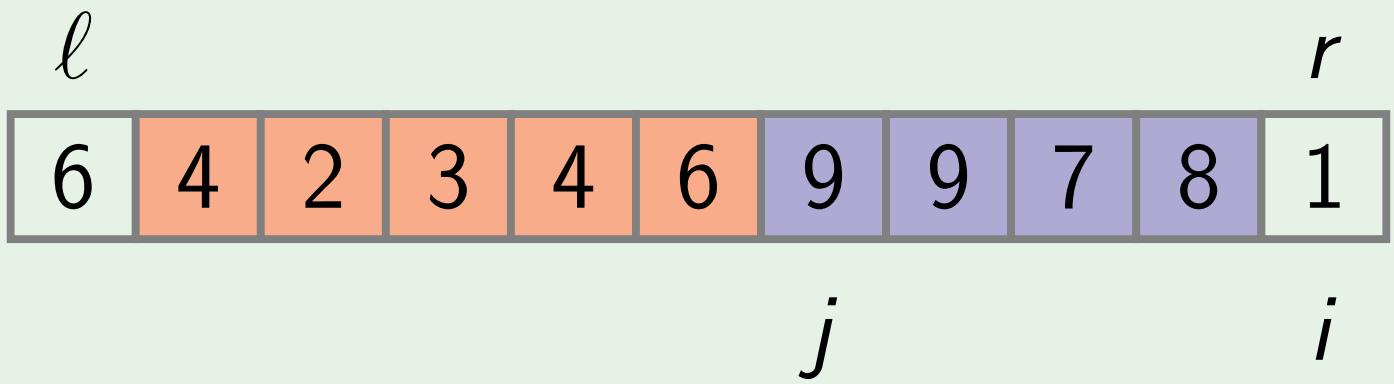
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



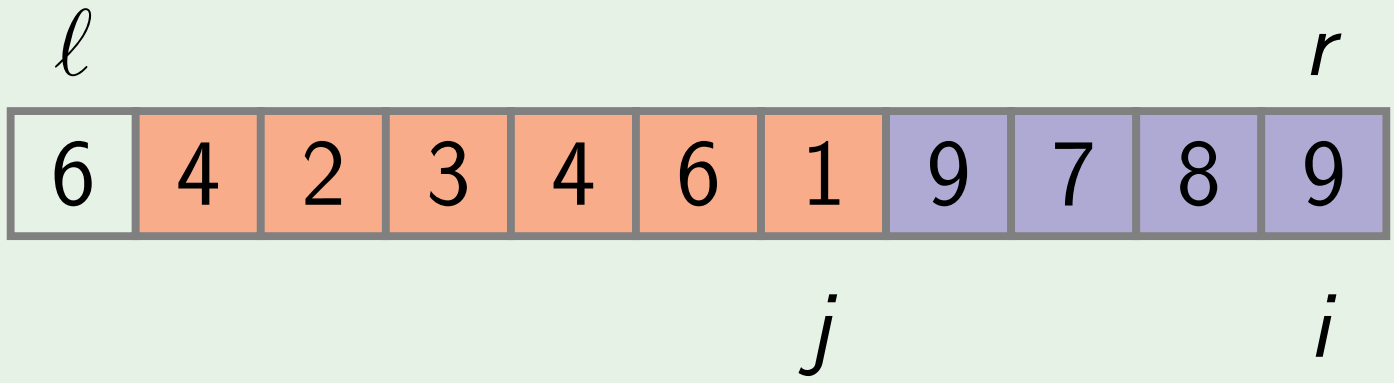
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$



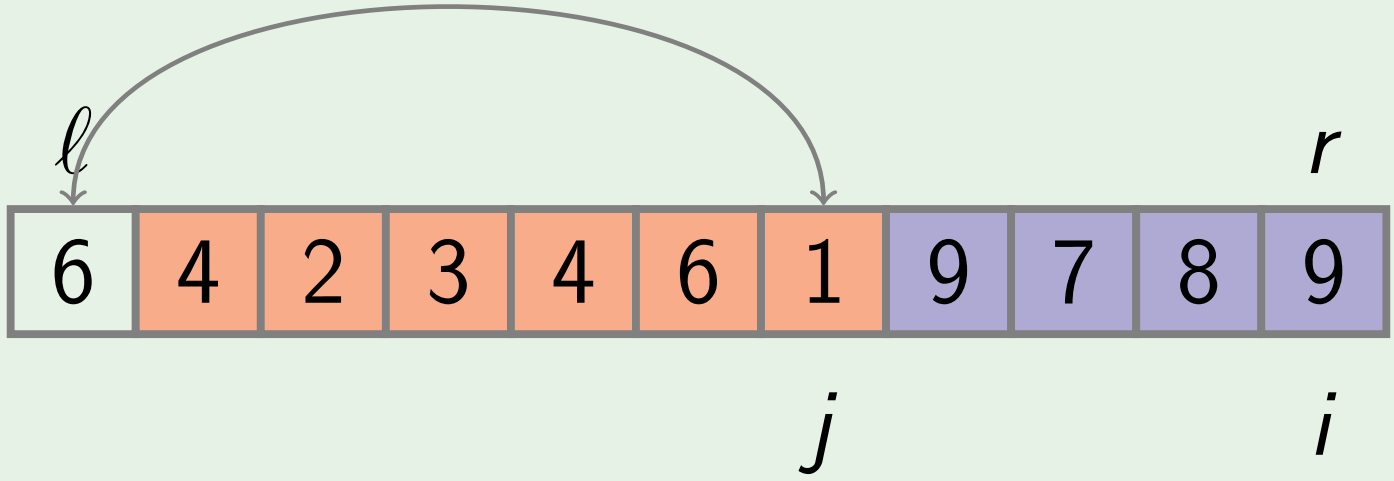
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$
- in the end, move $A[\ell]$ to its final place



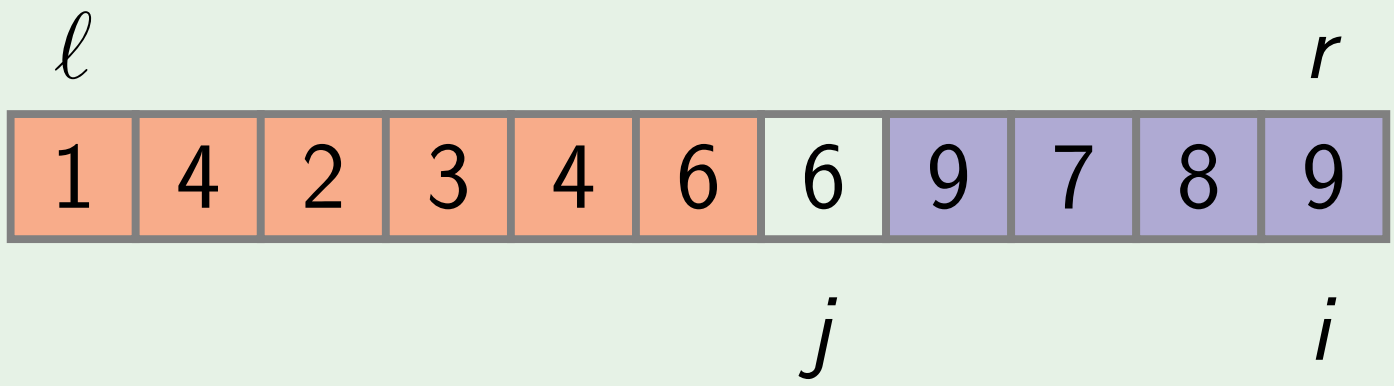
Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$
- in the end, move $A[\ell]$ to its final place



Partitioning: example

- the pivot is $x = A[\ell]$
- move i from $\ell + 1$ to r maintaining the following invariant:
  - $A[k] \leq x$ for all $\ell + 1 \leq k \leq j$
  - $A[k] > x$ for all $j + 1 \leq k \leq i$
- in the end, move $A[\ell]$ to its final place



Partition(A, ℓ, r)

$x \leftarrow A[\ell]$ {pivot}
$j \leftarrow \ell$
for i from $\ell + 1$ to r:
  if $A[i] \leq x$:
    $j \leftarrow j + 1$
    swap $A[j]$ and $A[i]$
  $\{A[\ell + 1 \ldots j] \leq x,\ A[j + 1 \ldots i] > x\}$
swap $A[\ell]$ and $A[j]$
return j

Outline

1. Overview
2. Algorithm
3. Random Pivot
4. Running Time Analysis
5. Equal Elements
6. Final Remarks

Unbalanced Partitions

- $T(n) = n + T(n-1)$:

$$T(n) = n+(n-1)+(n-2)+\cdots = \Theta(n^2)$$

Unbalanced Partitions

- $T(n) = n + T(n-1)$:

  $T(n) = n+(n-1)+(n-2)+\cdots = \Theta(n^2)$

- $T(n) = n + T(n-5) + T(4)$:

  $T(n) \geq n+(n-5)+(n-10)+\cdots = \Theta(n^2)$

Balanced Partitions

- $T(n) = 2T(n/2) + n$:

$$T(n) = \Theta(n \log n)$$

Balanced Partitions

- $T(n) = 2T(n/2) + n$:

$$T(n) = \Theta(n \log n)$$

- $T(n) = T(n/10) + T(9n/10) + n$:

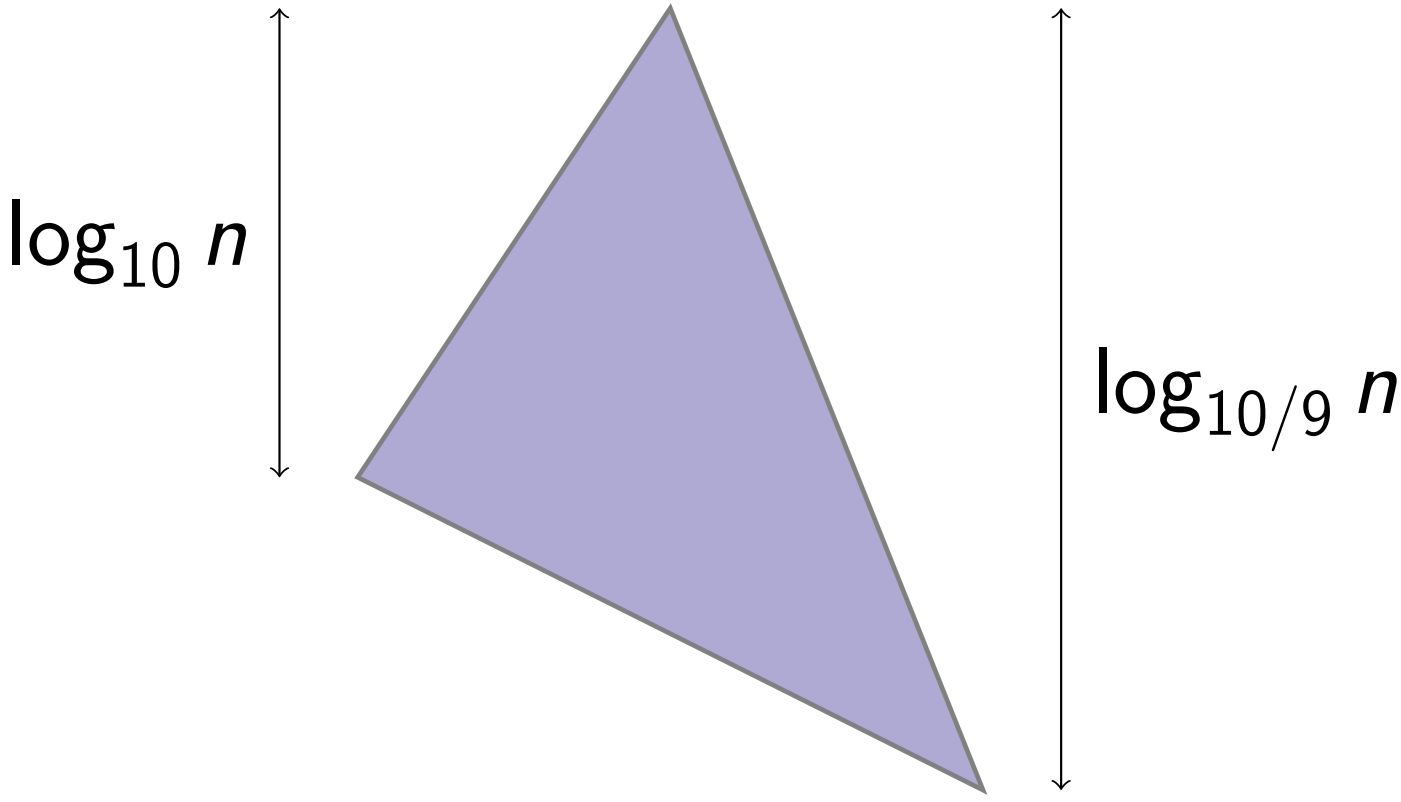
$$T(n) = \Theta(n \log n)$$

Balanced Partitions

$$T(n) = T(n/10) + T(9n/10) + O(n)$$

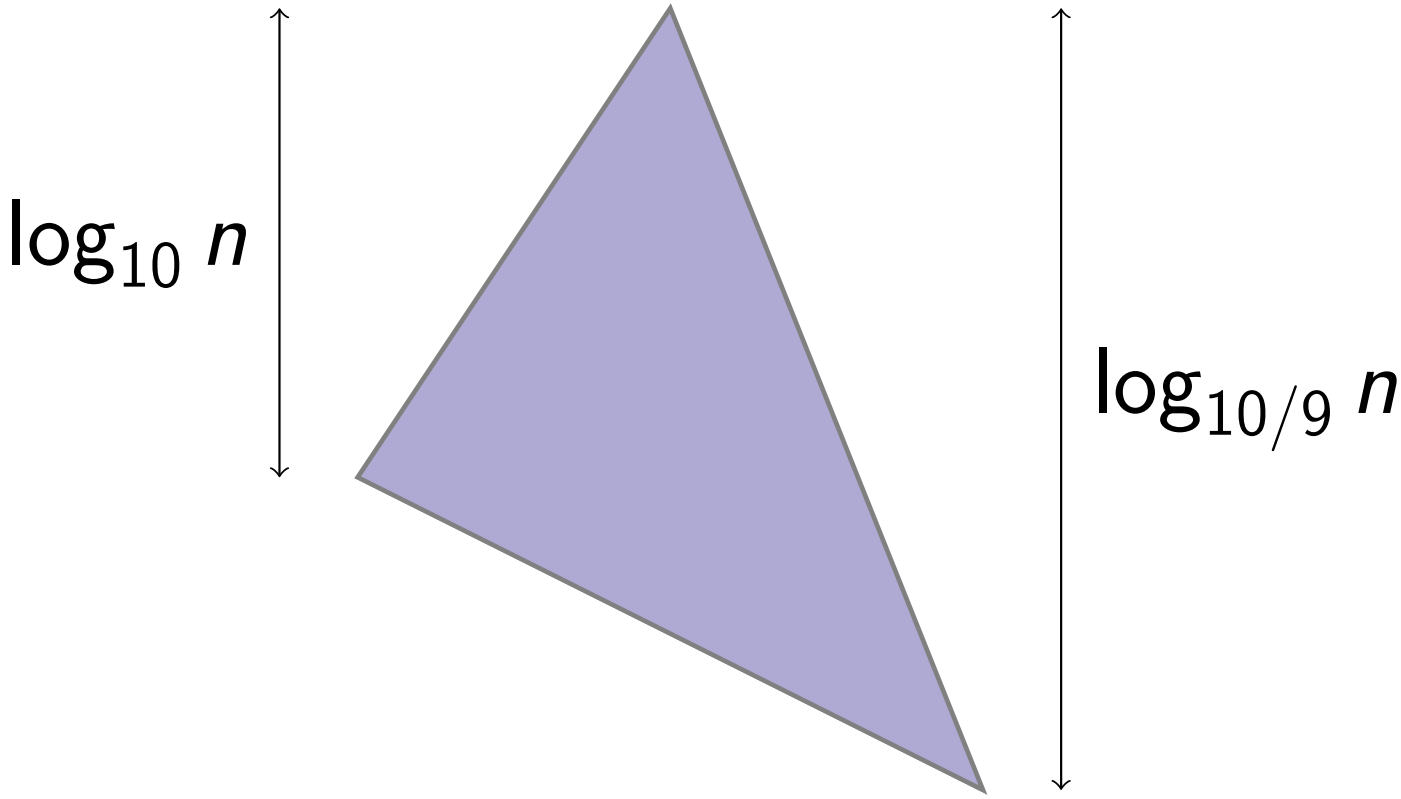
Balanced Partitions

$$T(n) = T(n/10) + T(9n/10) + O(n)$$



Balanced Partitions

$$T(n) = T(n/10) + T(9n/10) + O(n)$$



$$T(n) = O(n \log n)$$

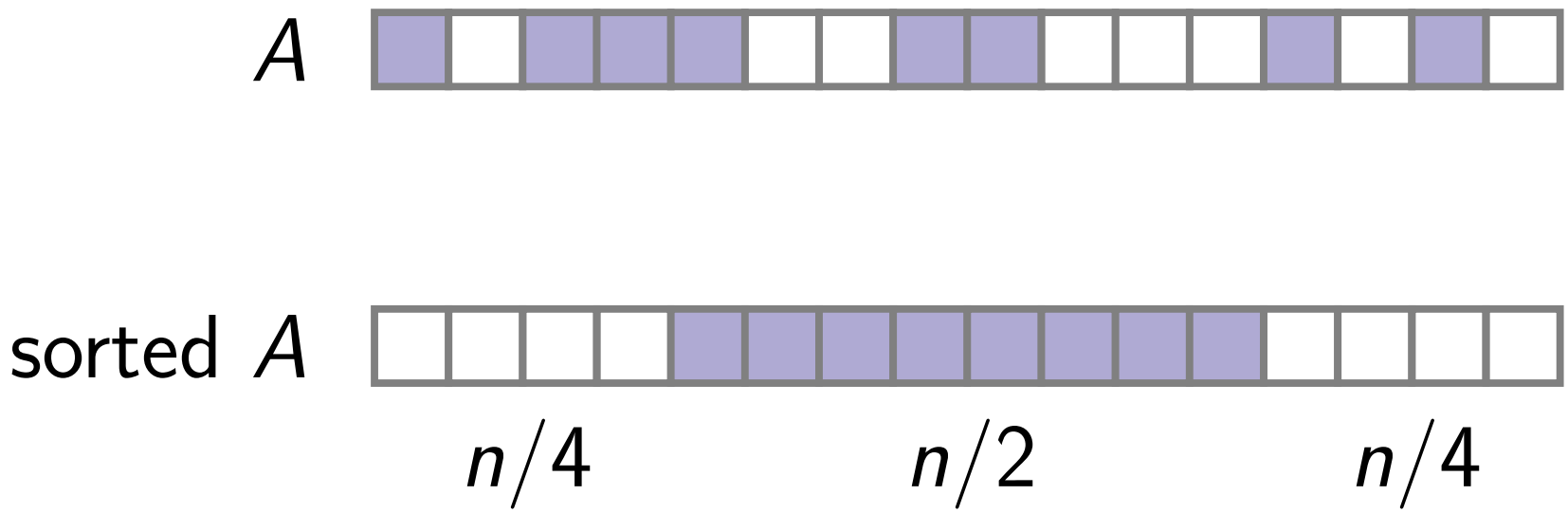
Random Pivot

RandomizedQuickSort(A, ℓ, r)

if $\ell \geq r$:
  return
$k \leftarrow$ random number between ℓ and r
swap $A[\ell]$ and $A[k]$
$m \leftarrow$ Partition(A, ℓ, r)
{$A[m]$ is in the final position}
RandomizedQuickSort$(A, \ell, m-1)$
RandomizedQuickSort$(A, m+1, r)$

Why Random?

half of the elements of A guarantees a balanced partition:



Theorem

Assume that all the elements of $A[1 \ldots n]$ are pairwise different. Then the average running time of `RandomizedQuickSort`(A) is $O(n \log n)$ while the worst case running time is $O(n^2)$.

Theorem

Assume that all the elements of $A[1 \ldots n]$ are pairwise different. Then the average running time of `RandomizedQuickSort`(A) is $O(n \log n)$ while the worst case running time is $O(n^2)$.

Remark

Averaging is over random numbers used by the algorithm, but not over the inputs.

Outline

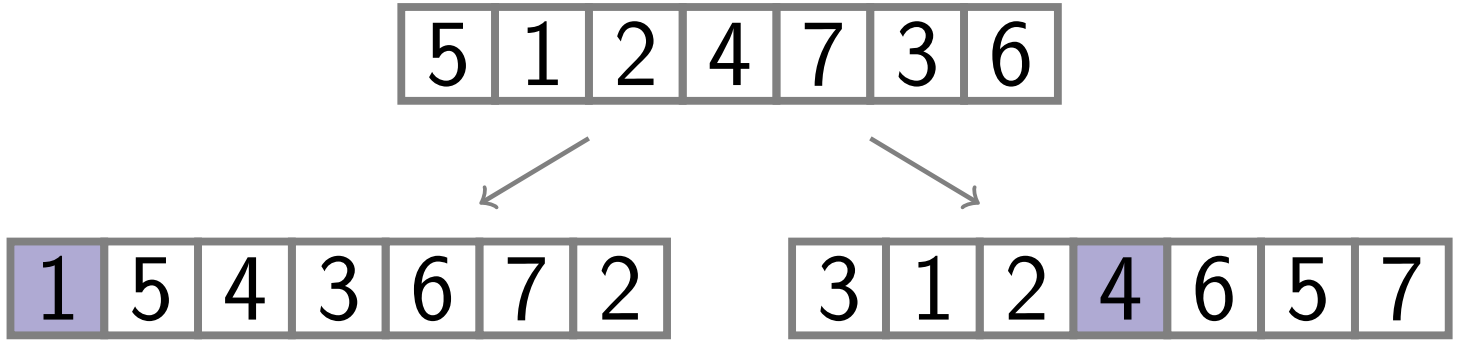
1. Overview
2. Algorithm
3. Random Pivot
4. Running Time Analysis
5. Equal Elements
6. Final Remarks

Proof Ideas: Comparisons

- the running time is proportional to the number of comparisons made

Proof Ideas: Comparisons

- the running time is proportional to the number of comparisons made
- balanced partition are better since they reduce the number of comparisons needed:



1 is min

$3, 1, 2 < 6, 5, 7$

Proof Ideas: Probability

A	5	1	8	9	2	4	7	3	6
A'	1	2	3	4	5	6	7	8	9

Proof Ideas: Probability

A	5	1	8	9	2	4	7	3	6
A'	1	2	3	4	5	6	7	8	9

Prob (1 and 9 are compared) $=$

Proof Ideas: Probability

A	5	1	8	9	2	4	7	3	6
A'	1	2	3	4	5	6	7	8	9

$$\text{Prob}\,(1 \text{ and } 9 \text{ are compared}) = \frac{2}{9}$$

Proof Ideas: Probability

A	5	1	8	9	2	4	7	3	6
A'	1	2	3	4	5	6	7	8	9

$$\text{Prob}\,(1 \text{ and } 9 \text{ are compared}) = \frac{2}{9}$$

$$\text{Prob}\,(3 \text{ and } 4 \text{ are compared}) =$$

Proof Ideas: Probability

A	5	1	8	9	2	4	7	3	6
A'	1	2	3	4	5	6	7	8	9

$$\text{Prob}\,(1 \text{ and } 9 \text{ are compared}) = \frac{2}{9}$$

$$\text{Prob}\,(3 \text{ and } 4 \text{ are compared}) = 1$$

Proof

- let, for $i < j$,

$$\chi_{ij} = \begin{cases} 1 & A'[i] \text{ and } A'[j] \text{ are compared} \\ 0 & \text{otherwise} \end{cases}$$

Proof

- let, for $i < j$,

$$\chi_{ij} = \begin{cases} 1 & A'[i] \text{ and } A'[j] \text{ are compared} \\ 0 & \text{otherwise} \end{cases}$$

- for all $i < j$, $A'[i]$ and $A'[j]$ are either compared exactly once or not compared at all (as we compare with a pivot)

Proof

- let, for $i < j$,

$$\chi_{ij} = \begin{cases} 1 & A'[i] \text{ and } A'[j] \text{ are compared} \\ 0 & \text{otherwise} \end{cases}$$

- for all $i < j$, $A'[i]$ and $A'[j]$ are either compared exactly once or not compared at all (as we compare with a pivot)
- this, in particular, implies that the worst case running time is $O(n^2)$

Proof (continued)

- crucial observation: $\chi_{ij} = 1$ iff the first selected pivot in $A'[i \ldots j]$ is $A'[i]$ or $A'[j]$

Proof (continued)

- crucial observation: $\chi_{ij} = 1$ iff the first selected pivot in $A'[i \ldots j]$ is $A'[i]$ or $A'[j]$
- then $\text{Prob}(\chi_{ij}) = \frac{2}{j-i+1}$ and $\text{E}(\chi_{ij}) = \frac{2}{j-i+1}$

Proof (continued)

Then (the expected value of) the running time is

$$\begin{aligned}
\mathrm{E}\sum_{i=1}^{n}\sum_{j=i+1}^{n}\chi_{ij} &= \sum_{i=1}^{n}\sum_{j=i+1}^{n}\mathrm{E}(\chi_{ij}) \\
&= \sum_{i<j}\frac{2}{j-i+1} \\
&\leq 2n\cdot\left(\frac{1}{2}+\frac{1}{3}+\ldots+\frac{1}{n}\right) \\
&= \Theta(n\log n)
\end{aligned}$$

Outline

1. Overview
2. Algorithm
3. Random Pivot
4. Running Time Analysis
5. Equal Elements
6. Final Remarks

Equal Elements

- what if all the elements of the given array are equal to each other?

Equal Elements

- what if all the elements of the given array are equal to each other?
- quick sort visualization

Equal Elements

- what if all the elements of the given array are equal to each other?
- quick sort visualization
- the array is always split into two parts of size 0 and $n - 1$

Equal Elements

- what if all the elements of the given array are equal to each other?
- quick sort visualization
- the array is always split into two parts of size 0 and $n - 1$
- $T(n) = n + T(n-1) + T(0)$ and hence $T(n) = \Theta(n^2)$!

To handle equal elements, we replace the line

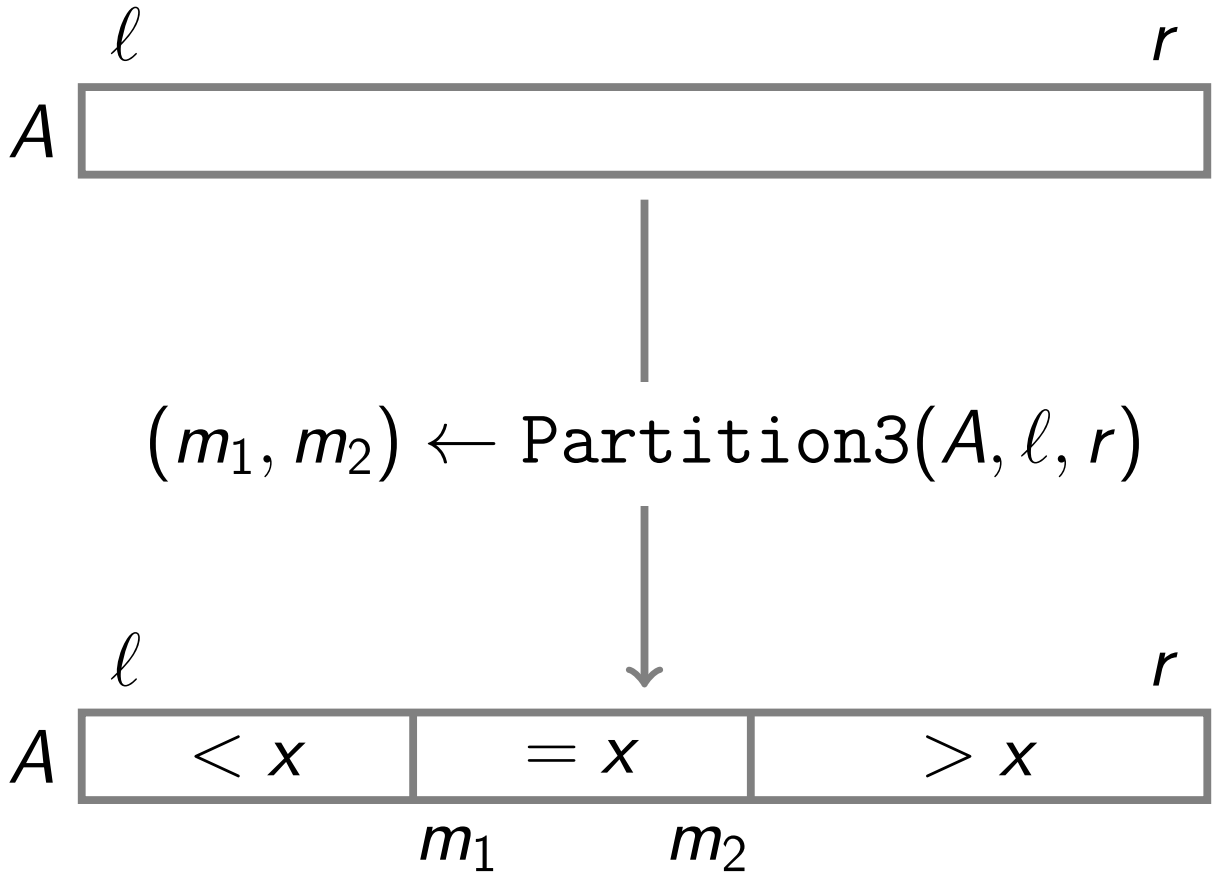
$$m \leftarrow \texttt{Partition}(A, \ell, r)$$

with the line

$$(m_1, m_2) \leftarrow \texttt{Partition3}(A, \ell, r)$$

such that

- for all $\ell \leq k \leq m_1 - 1$, $A[k] < x$
- for all $m_1 \leq k \leq m_2$, $A[k] = x$
- for all $m_2 + 1 \leq k \leq r$, $A[k] > x$


ℓ
r
A
$(m_1, m_2) \leftarrow \texttt{Partition3}(A, \ell, r)$
ℓ
r
A
$< x$
$= x$
$> x$
m_1
m_2

RandomizedQuickSort(A, ℓ, r)

if $\ell \geq r$:

  return

$k \leftarrow$ random number between ℓ and r

swap $A[\ell]$ and $A[k]$

$(m_1, m_2) \leftarrow$ Partition3(A, ℓ, r)

$\{A[m_1 \ldots m_2]$ is in final position$\}$

RandomizedQuickSort$(A, \ell, m_1 - 1)$

RandomizedQuickSort$(A, m_2 + 1, r)$

Outline

1. Overview
2. Algorithm
3. Random Pivot
4. Running Time Analysis
5. Equal Elements
6. Final Remarks

Tail Recursion Elimination

QuickSort(A, ℓ, r)

```
while ℓ < r:
    m ← Partition(A, ℓ, r)
    QuickSort(A, ℓ, m − 1)
    ℓ ← m + 1
```

QuickSort(A, ℓ, r)

```
while ℓ < r:
    m ← Partition(A, ℓ, r)
    if (m − ℓ) < (r − m):
        QuickSort(A, ℓ, m − 1)
        ℓ ← m + 1
    else:
        QuickSort(A, m + 1, r)
        r ← m − 1
```

QuickSort(A, ℓ, r)

```
while ℓ < r:
    m ← Partition(A, ℓ, r)
    if (m − ℓ) < (r − m):
        QuickSort(A, ℓ, m − 1)
        ℓ ← m + 1
    else:
        QuickSort(A, m + 1, r)
        r ← m − 1
```

Worst-case space requirement: $O(\log n)$

Intro Sort

- runs quick sort with a simple deterministic pivot selection heuristic (say, median of the first, middle, and last element)

Intro Sort

- runs quick sort with a simple deterministic pivot selection heuristic (say, median of the first, middle, and last element)
- if the recursion depth exceeds a certain threshold $c \log n$ the algorithm switches to heap sort

Intro Sort

- runs quick sort with a simple deterministic pivot selection heuristic (say, median of the first, middle, and last element)
- if the recursion depth exceeds a certain threshold $c \log n$ the algorithm switches to heap sort
- the running time is $O(n \log n)$ in the worst case

Conclusion

- Quick sort is a comparison based algorithm

Conclusion

- Quick sort is a comparison based algorithm
- Running time: $O(n \log n)$ on average, $O(n^2)$ in the worst case

Conclusion

- Quick sort is a comparison based algorithm
- Running time: $O(n \log n)$ on average, $O(n^2)$ in the worst case
- Efficient in practice